

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

GROUP 3 SC2006 - SCSD

LAB 5 DELIVERABLES

Team Members
1. Vivek Shrey (U2440609F) - Team Leader
2. Ching Xin Wei (U2421032K)
3. Dharshini Saravanan (U2420758E)
4. Li Sinuo (U2421736H)
5. Sivasankar Sivagnana Sundari (U2420528F)

Table of Contents

The internal hyperlinks in the Table of Contents work correctly when the PDF is opened directly (e.g., after downloading it).

1. Introduction
    1.1 Project Mission Statement
    1.2 Functional Requirements
    1.3 Non-Functional Requirements
    1.4 Data Dictionary
2. Overall Description
    2.1 Use Case Diagram
        2.1.1 Use-Case Descriptions
    2.2 Design Model
        2.2.1 Relationship Between Boundary, Control and Entity Classes
        2.2.2 Class Diagram
            2.2.2.1 User Interfaces (Boundary Classes)
            2.2.2.2 Controllers (Control Classes)
            2.2.2.3 App Object Layer (Entity Classes)
        2.2.3 Sequence Diagrams
            2.2.3.1 Use Case - UC-2 (Login)
            2.2.3.2 Use Case - UC-3 (Manage Profile and Skills)
            2.2.3.3 Use Case - UC-5 (View Recommended Jobs)
            2.2.3.4 Use Case - UC-7 (View Job Details)
        2.2.4 Dialog Map Diagram
    2.3 System Architecture
        2.3.1 Presentation
        2.3.2 JobHackSG App Logic Layer:
        2.3.3 JobHackSG Object Layer
        2.3.4 Persistent Data Layer
    2.4 Application Skeleton
    2.5 Operating Environment
3. UI Mockups
    3.1 Onboarding
    3.2 For You Page
    3.3 Search Page
    3.4 Job Details
    3.5 Applications
    3.6 Profile
4. Testing
    4.1 Black Box Testing
        4.1.2 AuthenticationController
        4.1.2 Equivalence Class and Boundary Value Testing
        4.1.3 Test Cases and Results
            4.1.3.1 Login
            4.1.3.2 Register

4.2 White Box Testing
4.2.1 Login
4.2.1.1 Control Flow Graph
4.2.1.2 Basis Path Testing
4.2.1.3 Test Cases and Results
4.2.2 Registration
4.2.2.1 Control Flow Graph
4.2.2.2 Basis Path Testing
4.2.2.3 Test Cases and Results
Appendix A: Analysis Models

1. Introduction

This document will outline the software requirements for our project “Job Hack SG”, including the functionalities, features and constraints of our system, aiming to provide a clear and comprehensive guide for development. This document is integral to the guiding of development, testing and any future maintenance of the system, making sure that it aligns with the system objectives we have set out.

1.1 Project Mission Statement

Job Hack SG helps students and early-career jobseekers discover *where demand is growing* in Singapore and *which roles match their skills today*. The app combines sector insights (growth, vacancies, in-demand skills) with a personal matching engine over live job postings. Users can search and filter jobs, see matches based on their current skill profile, and view missing-skill gaps alongside suggested courses to upskill. An applications page allows them to apply to that particular role by routing them to the correct website. The goal is a clear, testable, end-to-end prototype that demonstrates sector specific guidance along with practical role recommendations, built primarily on public datasets/APIs, specific to the Singapore context. What sets Job Hack SG apart from other competing platforms is that this will be a one stop portal for career related matters, and will cater specially for Singapore.

1.2 Functional Requirements

This outlines the functional requirements of our project to meet the goals of our mission statement above.

1. *The system shall allow users to create an account and add on their current skills to tune their job profile.*

    1.1. The system shall let users register and sign in with email/password.

    1.2. The system shall allow users, once signed - in, to edit profile fields: name, school, programme, graduation year.

    1.3. The system shall let users add, edit, delete skills (name; proficiency 1–5).

    1.4. The system also allows the user to do a Resume upload to ease the process of manually entering skills.

2. ***The system shall be updated constantly with job related data.***

    2.1. The system shall ingest job postings daily (title, company, location in Singapore, description, tags/skills, posting date, apply URL, etc).

    2.2. The system shall compute and store a posting's active status if posted ≤ 60 days ago.

    2.3. The system shall ingest sector/skills summaries (e.g., vacancies by sector, top skills by sector) and store "sector → hot skills" pairs.

3. ***The system shall display a Job Trends Dashboard.***

    3.1.The system shall display a Job Sector Trends dashboard: list sectors with indicators (e.g., vacancy index, growth direction) and their top skills.

    3.2. The system shall allow users to pick a sector and view top roles and top skills for that sector.

4. ***The system shall have the ability of allowing users to search for jobs relevant to them.***

    4.1.The system shall allow users to do a keyword search over title/company/description and filters (location in Singapore, job type, recency).

    4.2. The system shall display a Matched Skills and Missing Skills chips for each recommended job.

5. Course suggestions shall be made available to the users.

5.1. For each job's top Missing Skills, the system shall display up to 3 relevant courses (title, provider, URL) for the user, that they can take up to improve their skillset.

6. The system shall allow users to see details of jobs after searching for it.

6.1. The system shall display a Job Details page with a full job description.

6.2. The system shall allow users to save/unsave a job.

6.3. The system shall allow users to apply for the posting by clicking on the link on the portal. It will route them to the company's website.

1.3 Non-Functional Requirements

This outlines the non-functional requirements to ensure a good, functional and reliable product for our users.

1. *Usability*

    1.1. The system shall be usable by both first-time and regular users.

        1.1.1. The system shall allow a first-time user to complete onboarding (registration and add at least three skills) within 5 minutes without external help.

        1.1.2. The system shall ensure all interactive UI components are fully keyboard - navigable.

2. *Performance*

    2.1. The system shall load the home/dashboard page within 3 seconds under normal network conditions.

    2.2. The system shall handle up to 500 concurrent users without degradation of service.

    2.3. The system shall return search results in under 2 seconds for datasets containing ≤ 10,000 job postings.

    2.4. The system shall generate and display personalized recommendations within 3 seconds for a user profile containing ≤ 40 skills.

3. *Security*

    3.1. The system shall store user passwords using secure hashing and salting with an industry-standard algorithm.

    3.2. The system shall automatically log out inactive users after 30 minutes.

    3.3. The system shall virus scan all personally uploaded files before storage.

    3.4. The system shall store uploaded files with restricted access permissions to prevent unauthorized usage by any third parties.

4. *Reliability and Availability*

4.1. The system shall have an uptime of at least 99% between 09:00 and 23:00 SGT daily excluding scheduled maintenance, which shall be announced beforehand.

4.2. The system shall recover from server failures within 5 minutes.

4.3. The system shall ensure no data loss in case of unexpected downtime by performing daily backups.

4.4. The system shall complete nightly ingestion jobs successfully on ≥ 95% of calendar days.

4.5. The system shall generate an admin alert in case of ingestion failure.

5. *Maintainability*

5.1. The system shall be designed in separate layers (front-end and back-end) so that changes in one do not break the other.

5.2. The core skill-matching logic shall achieve ≥ 60% unit test coverage by Lab 4.

5.3. The system shall allow updates to the data ingestion and matching components with less than 1 hour of downtime.

1.4 Data Dictionary

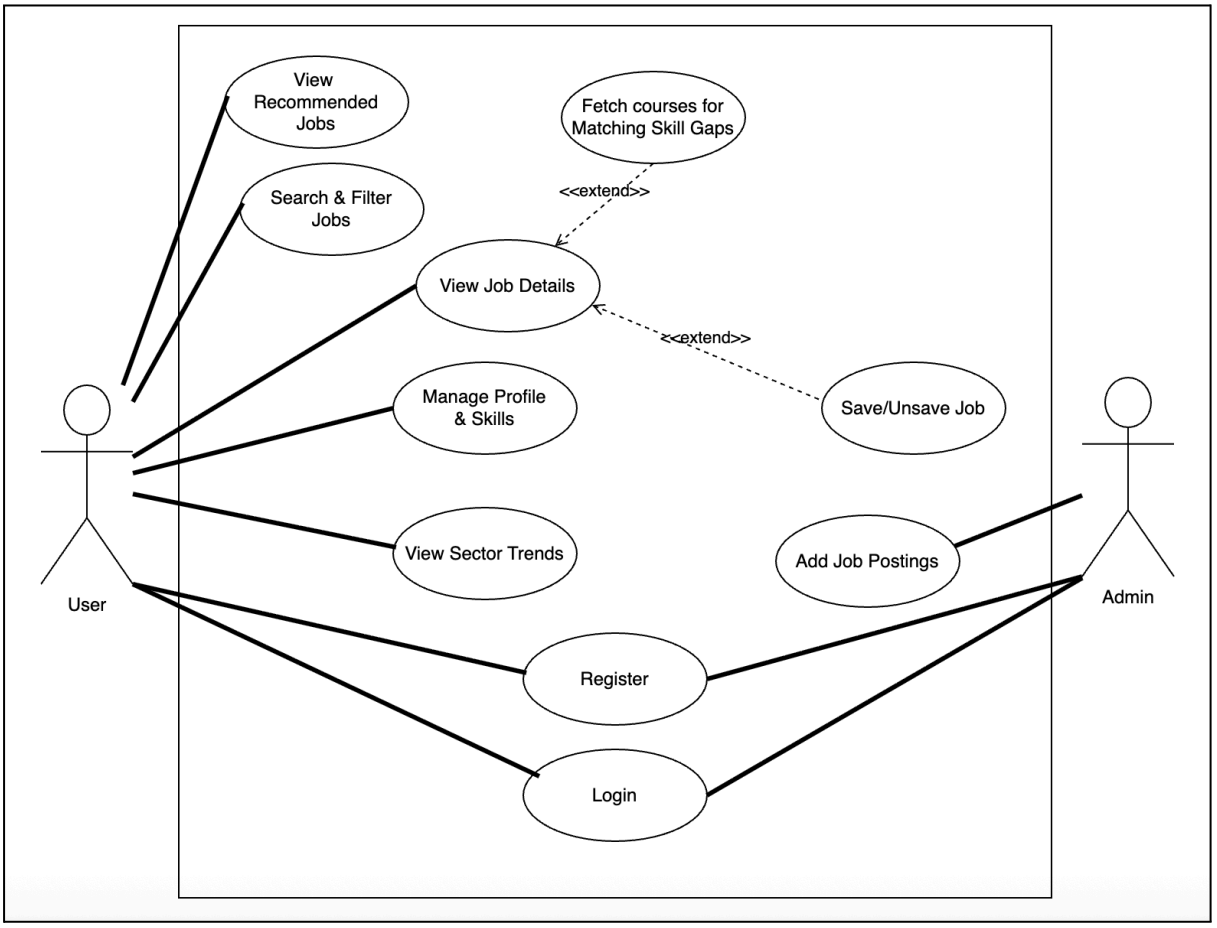
This outlines the structure, content, and variable definitions for our project dataset.

Name	Definition
User	A student or new jobseeker using JobHackSG. A user maintains a profile with personal details and skills and receives personalised information on sector trends and job recommendations.
Skill	A specific capability of a user (e.g., "Python Programming," "Data Analysis") that can be listed in their profile. It can be extracted from resumes, or identified as popular within a sector.
Resume	A digital document uploaded by the user that provides a description of their experience, educational qualifications, and skills. The system analyses the resume to pick out user skills and helps with profile building.
JobPosting	A structured representation of a job role sourced from live job boards or APIs. It includes the job title, employer, job description, location, required skills, and application link.
Sector	A labour market category in Singapore (e.g., ICT, Healthcare, Finance) with information such as growth trends, vacancies, and emerging skill demands.
SectorHotSkill	A skill classified as popular within a specific sector which can be identified from job posting analytics or public datasets. It is highlighted to users as a highly valued competency for better employability.
Recommendation	A personalised suggestion generated for a user, either a job or course, based on alignment of their skills with job/sector requirements. It includes skill gaps, and upskilling opportunities.
Course	An upskilling or training program (e.g., SkillsFuture, university modules) related to required skills in JobPostings. It is suggested to users to close identified skill gaps.
Application	A record of a user’s action to apply for a JobPosting. It is used to track when and where job applications are made and directs the user to the employer’s or application page.
IngestionRun	A logged process that retrieves, cleans, and updates job postings and sector data from external sources. It records metadata such as execution time, source, success/failure status, and error details.

2. Overall Description

This specifies the project's overall descriptions and requirements, including its use case diagram and descriptions, class diagram, sequence diagrams, dialog map and system architecture. This helped us plan out the implementation and design of our product.

2.1 Use Case Diagram



Actors

- **User** – a job seeker who browses, searches, manages profile/skills, uploads resume, saves jobs, logs applications, and views recommendations/trends.
- **Admin** – an administrator that adds new job postings to the system

Potential Narrative for a user:

A **User** can **Register** (UC-1) then **Login** (UC-2). Once authenticated, they can **Manage Profile & Skills** (UC-3) to improve recommendations. They can explore **Sector Trends** (UC-4) or directly **View Recommended Jobs** (UC-5) and **Search & Filter Jobs** (UC-7). Both lead to **View Job Details** (UC-8). From **Job Details**, the user may **Save/Unsave** (UC-9), **Log/Update Application** (UC-10), or **Fetch Courses for Matching Skill Gaps** (UC-11). This way, the user helps themselves find a job - inclining to their interests.

Use Cases

1. **UC-1 Register**
2. **UC-2 Login**
3. **UC-3 Manage Profile & Skills**
4. **UC-4 View Sector Trends**
5. **UC-5 View Recommended Jobs**
6. **UC-6 Search & Filter Jobs**
7. **UC-7 View Job Details**
8. **UC-8 Save/Unsave Job**
9. **UC-9 Fetch Courses for Matching Skill Gaps**
10. **UC-10 Add Job Postings**

Include / Extend Relationships

- **Extends**
    - **UC-9 Save/Unsave Job** «extend» **UC-8 View Job Details** (optional bookmarking from the details page).
    - **UC-10 Log/Update Application** «extend» **UC-8 View Job Details** (optional apply/log action from details).
    - **UC-11 Fetch Courses for Matching Skill Gaps** «extend» **UC-8 View Job Details** (optional upskilling suggestions from details).

2.1.1 Use-Case Descriptions

Use Case ID:	**UC-1**		
Use Case Name:	**Register**		
Created By:	Sundari	Last Updated By:	Sundari
Date Created:	12 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	This is practically the first step for a new user to enter the application.
Preconditions:	The application is started and User is prompted to register.
Postconditions:	User has a newly created account and are logged in OR User is given a valid explanation for why their registration failed.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. System displays fields for registration email and password. 2. User enters their details into the field. 3. User presses the Register button. 4. System verifies their email and password and creates a new account for them in the system database. 5. If the account is created, system displays a message that says ‘Registration is successful!’ 6. User is brought to the home page / landing screen.
Alternative Flows:	**AF-S1:** If User leaves the field blank 1. System highlights missing fields with error message “This field is required” **AF-S2:** If the email or password is entered incorrectly 1. The system displays a message saying ‘Invalid email or password not matching criteria! Please Try Again!”

	2. Return to Step 1.
Exceptions:	**EX-1:** System cannot reach database → Error: “Registration service temporarily unavailable”. **EX-2:** Network disconnected → Error: “Please check your internet connection”.
Includes:	None
Special Requirements:	Passwords must be stored with secure hashing + salting
Assumptions:	User has access to their own email
Notes and Issues:	None

Use Case ID:	**UC-2**		
Use Case Name:	**Login**		
Created By:	Vivek Shrey	Last Updated By:	Vivek Shrey
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	This is practically the first step for an existing user to enter the application.
Preconditions:	The user already has an existing account, and has been authenticated previously.
Postconditions:	User has logged into their account OR User is given a valid explanation for why their login failed.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. System displays fields for email and password 2. User enters their details into the field. 3. User presses the login button. 4. System verifies their email and password with the account information in the system database. 5. If the account is verified, system displays a message that says ‘Login is successful!’ 6. User is brought to the home page / landing screen.
Alternative Flows:	**AF-S1:** If the email or password is entered incorrectly 1. The system displays a message saying ‘Invalid email or password! Please Try Again!” 2. Return to Step 3. **AF-S2:** If User leaves the field blank

	1. System highlights missing fields with error message “This field is required”
Exceptions:	**EX-1:** System cannot reach database → Error: “Login service temporarily unavailable”. **EX-2:** Network disconnected → Error: “Please check your internet connection”.
Includes:	None
Special Requirements:	Passwords must be stored with secure hashing + salting
Assumptions:	User has access to their own email
Notes and Issues:	None

Use Case ID:	**UC-3**		
Use Case Name:	**Manage Profile and Skills**		
Created By:	Vivek Shrey	Last Updated By:	Vivek Shrey
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	This is the section where users will update their profile and update their current skills that are relevant for their job hunt.
Preconditions:	User is Authenticated and logged in.
Postconditions:	System displays the updated profile and skills.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. User clicks on profile. 2. System displays any existing profile tied to the user, displaying the profile and skills 3. User clicks the option to manage profile and skills. 4. System displays the profile and skill fields that can be edited. 5. User enter their changes for the fields and confirms their changes. 6. System verifies these changes and confirms these changes to its database, displaying a message to the user “Your profile has been successfully changed!”
Alternative Flows:	**AF-S1:** If any of User’s profile fields is empty 1. System displays an empty profile field, with a message that says “You have not added anything to this field! Add to it now!” 2. Return to Step 3.

	AF-S2: If User enters invalid skill proficiency (e.g. <2) 1. System rejects and highlights error **AF-S3:** If User attempts to delete all skills 1. System displays warning: “At least one skill must be present for recommendations.”
Exceptions:	**EX-1:** Profile update fails due to database error → System displays “Update failed. Please try again later.”
Includes:	None
Special Requirements:	Changes should sync across all modules within 5 seconds.
Assumptions:	None
Notes and Issues:	None

Use Case ID:	**UC-4**		
Use Case Name:	**View Sector Trends**		
Created By:	Xin Wei	Last Updated By:	Xin Wei
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	The Trends section allows users to see the current trends in the market, specific to their interested job's domain.
Preconditions:	Authenticated; sector data exists in the database.
Postconditions:	System displays the data list of the selected sector.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. User opens the "Trends" section. 2. System lists sectors based on vacancy/growth and top skills. 3. User selects a sector. 4. System checks its database for the top roles and most suited skills for that sector. 5. System displays the top roles and most suited skills for that sector.
Alternative Flows:	**AF-S1**: If there is no data in the selected sector 1. System displays a blank field with a message saying "No data found!" 2. Return to Step 2. **AF-S2:** Sector data partially missing (e.g. no top skills) 1. System displays available data with placeholder text for missing parts.

Exceptions:	**EX-1:** External jobs API unavailable → trends cannot load → System displays fallback message “Trends temporarily unavailable”
Includes:	None
Special Requirements:	Trends data must update at least once daily.
Assumptions:	User has at least one valid sector available in Database
Notes and Issues:	Future version may support filtering trends by location/industry tags

Use Case ID:	**UC-5**		
Use Case Name:	**View Recommended Jobs**		
Created By:	Xin Wei	Last Updated By:	Xin Wei
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	Users can see suggested jobs that are ideal for them based on their requirements and skills.
Preconditions:	Authenticated; existing job postings, user has at least 1 skill.
Postconditions:	System displays ranked jobs filtered by the skills.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. User selects “For You”. 2. System computes a match with matched/missing skills. 3. System displays a list of jobs recommended for user. 4. User scrolls through the list of jobs to select which one they like. 5. User selects a job. 6. See **UC-7**.
Alternative Flows:	**AF-S1:** If no jobs are matched to the skills OR User has no skills 1. System displays a message “You have not added any skills!” 2. System displays generic trending jobs below the message.
Exceptions:	**EX-1:** Recommendation engine fails → fallback: display most popular jobs by posting frequency **EX-2:** Jobs API response timeout
Includes:	None

Special Requirements:	Match score must be displayed for each job
Assumptions:	Recommendation engine can run in <3 seconds for <40 skills
Notes and Issues:	In the future, may integrate ML-based personalisation beyond skill matching

Use Case ID:	**UC-6**		
Use Case Name:	**Search and Filter Jobs**		
Created By:	Xin Wei	Last Updated By:	Xin Wei
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	This section allows users to search for jobs based on keywords through filtering based on skills, salary, date posted, etc.
Preconditions:	Authenticated, existing job postings.
Postconditions:	System displays results reflecting query/filters.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. System displays list of unfiltered jobs based on date updated. 2. User opens the search / filter option box. 3. User selects and enters filters or searches based on key words 4. System searches database for jobs matching filters / key words 5. System displays to User list of jobs based on filters / key words. 6. User selects a job. 7. See **UC-7**.
Alternative Flows:	**AF-S1:** User enters invalid filter values (e.g. salary range min > max) 1. System prompts the user to correct input. **AF-S2:** If there is no available jobs based on the key word / filter

	1. System displays a message saying “No jobs matched! Use another keyword or select less filters” 2. Return to Step 2.
Exceptions:	**EX-1:** Search service unavailable. **EX-2:** Slow API → system times out and shows “search unavailable, please retry”.
Includes:	None
Special Requirements:	Search results should appear within 2 seconds under normal load
Assumptions:	Job postings exist in database
Notes and Issues:	Might need to cache frequently searched queries for faster response

Use Case ID:	**UC-7**		
Use Case Name:	**View Job Details**		
Created By:	Dharshini	Last Updated By:	Dharshini
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	This section allows users to view the details of the job that has either interested them/appeals most to them based on their skills.
Preconditions:	User is authenticated; job is active.
Postconditions:	System displays the details of the job; user may save/log application.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. User clicks on job. 2. System checks the database for the job details. 3. System displays the job details. 4. User can apply for the job. See **UC-10**.
Alternative Flows:	**AF-S1:** if user does not wish to apply but to view job later 1. User can save jobs as a bookmark to the Saved section. 2. See **UC-8**. 3. User clicks on the “Saved” button at the bottom. 4. System displays the saved jobs. **AF-S2:** if job is removed 1. System displays a message saying “Job has been removed!” 2. Returns back, depending on the previous page User was on.

Exceptions:	**EX-1:** Job details partially missing (e.g. no salary info) → show available details with “N/A” where missing.
Includes:	Extended by UC-9, UC-10, UC-11
Special Requirements:	Must show Apply button linking to external site
Assumptions:	Selected job is active in database
Notes and Issues:	details page must handle rich text job descriptions (bullet points, links)

Use Case ID:	**UC-8**		
Use Case Name:	**Save/Unsave Job**		
Created By:	Dharshini	Last Updated By:	Dharshini
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	The user has the option to save or unsave the job based on their interest.
Preconditions:	User is Authenticated; User is on an existing Job Detail page and the job is active.
Postconditions:	System stores / removes the selected job in the Saved section.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. On the job's details page, User clicks on the bookmark button at the upper right corner. 2. System stores the selected job in the "Saved" section.. 3. User goes to the saved jobs page. 4. System displays saved jobs to user. 5. User clicks on a saved job to go its detail page. 6. User again clicks on the bookmark button. 7. System removes the selected job in the "Saved" section.
Alternative Flows:	**AF-S1**: if job has been removed 1. System displays a message saying "Job cannot be saved, it has been removed!" when user tries to save it. 2. Return to Step 1. **AF-S2:** if job has already been saved 1. System changes button state to "Unsave"

Exceptions:	**EX-1:** Database save operation fails → system shows error “unable to save job. please retry.”
Includes:	Extends UC-8
Special Requirements:	Saved jobs must persist across user sessions
Assumptions:	User has at least one valid saved job entry
Notes and Issues:	Consider limiting max saved jobs (e.g. 100) to reduce DB overhead.

Use Case ID:	**UC-9**		
Use Case Name:	**Fetch Courses for Matching Skill Gaps**		
Created By:	Sundari	Last Updated By:	Sundari
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	Users can see courses that will help them upskill and make them better suited for the job that they potentially want to apply for.
Preconditions:	User is Authenticated; User has existing skills in their profile, is on an existing Job Detail page and the job is active.
Postconditions:	Display the courses to supplement skill gaps.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. User clicks on the option to view courses for skill gaps. 2. System checks the database for matching courses to supplement skill gaps. 3. System displays any results and corresponding links to the User. 4. User can choose to click on the links to the courses.
Alternative Flows:	**AF-S1:** if no courses available for skill gaps 1. System displays a message saying that “There are no available courses!” 2. Return to the Job Details page. **AF-S2:** multiple courses overlap 1. System shows top 3 courses ranked by provider credibility/relevance.

Exceptions:	**EX-1:** Courses API unavailable → system shows “courses service temporarily unavailable”.
Includes:	Extends UC-8
Special Requirements:	Course links must open in external browser
Assumptions:	Courses API has at least some entries for common skill gaps
Notes and Issues:	Future integration with skillsfuture APIs directly for real-time funding info.

Use Case ID:	**UC-10**		
Use Case Name:	**Add Job Postings**		
Created By:	Li Sinuo	Last Updated By:	Li Sinuo
Date Created:	07 - 11 - 2025	Date Last Updated:	08 - 11 - 2025

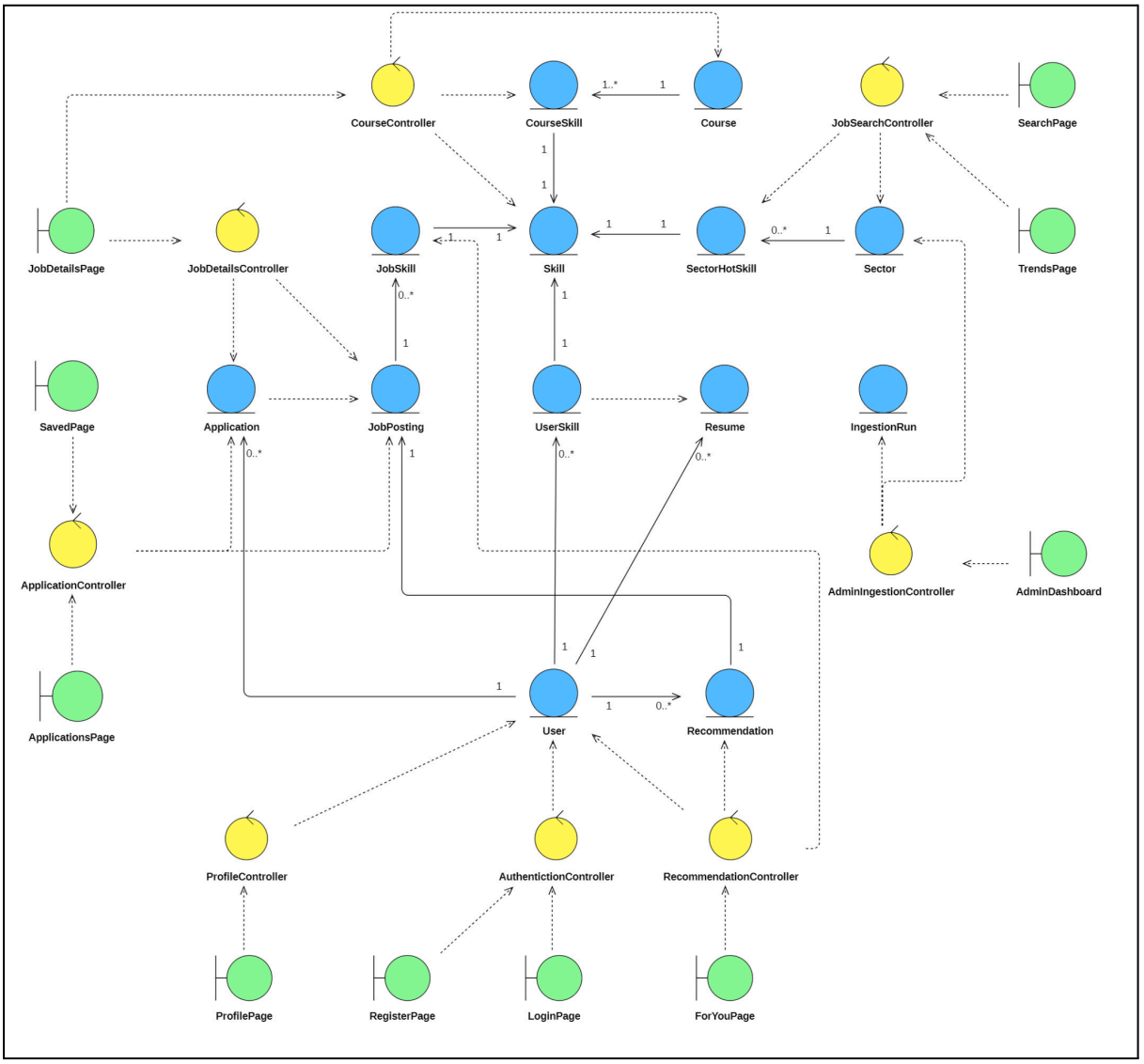
Actor:	Admin
Description:	This is done by the admi. This is to add any new job postings to the database.
Preconditions:	Admin Authenticated. Available job postings to be added.
Postconditions:	New job postings are added to the database.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. Admin checks the job postings database. 2. Admin adds the new job postings to the database. 3. System confirms jobs posting details and adds to the database. 4. System displays new job postings on all the lists to the user.
Alternative Flows:	**AF-S1:** if new jobs postings have missing fields or error 1. System displays a message indicating “Error! New job posting has missing or invalid details!” 2. Admin fixes any mistakes on the job posting and adds it to the system again. 3. Return to step 3. **AF-S2:** duplicate posting detected 1. System warns and blocks insert
Exceptions:	**EX-1:** Database write error prevents saving posting.
Includes:	None

Special Requirements:	Job postings must include mandatory fields (title, company, location, posting date)
Assumptions:	Admin has authority to insert postings
Notes and Issues:	May require moderation for inappropriate/invalid postings.

2.2 Design Model

This includes the diagrams and details we have for our project's design model, allowing us to have a better idea and planning of the implementation of our product.

2.2.1 Relationship Between Boundary, Control and Entity Classes



(clearer image uploaded directly to GitHub)

Key:

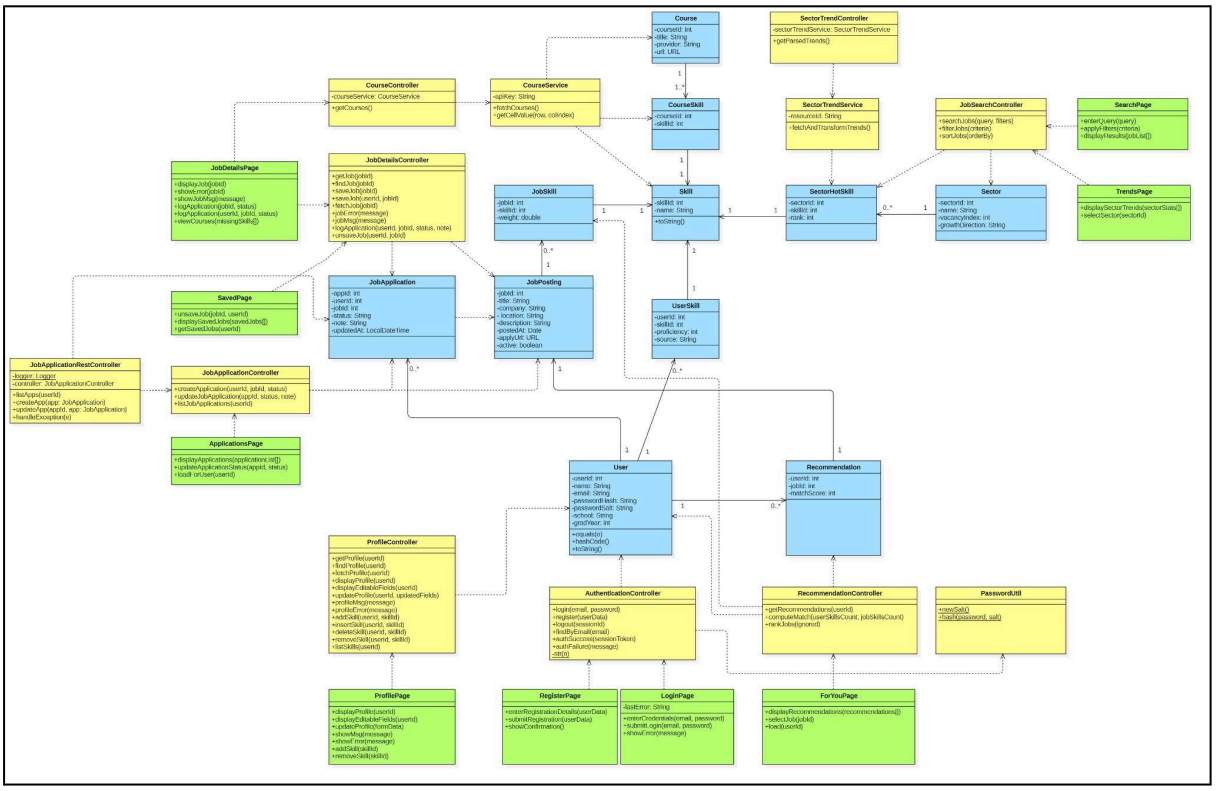
Green - Boundary Classes

Yellow - Control Classes

Blue - Entity Classes

2.2.2 Class Diagram

If the image is unclear, please refer to the separate file on GitHub.



Key:

Green - Boundary Classes

Yellow - Control Classes

Blue - Entity Classes

2.2.2.1 User Interfaces (Boundary Classes)

- **SearchPage** – enter a query, apply filters, display paged results; uses `JobSearchController`.
- **TrendsPage** – browse sector trends and vacancy index; uses `JobSearchController`.
- **JobDetailsPage** – show job info, save/unsave, apply; uses `JobDetailsController`.
- **SavedPage** – list saved jobs; uses `ApplicationController`.
- **ApplicationsPage** – list and update application statuses; uses `JobApplicationController`.
- **ProfilePage** – view/edit profile, skills, resume upload; uses `ProfileController`.
- **ForYouPage** – show ranked recommendations; uses `RecommendationController`.
- **LoginPage / RegisterPage** – auth flows; use `AuthenticationController`.

2.2.2.2 Controllers (Control Classes)

- **JobSearchController**
  `searchJobs(query, filters)`; `sortJobs(orderBy)`; fetches jobs by sector/keywords and vacancy index; consults `Sector`, `JobPosting`.
- **JobDetailsController**
  `getJob(jobId)`; `saveJob(userId, jobId)`; `unsaveJob(...)`; `applyToJob(...)`; logs message and status updates; touches `JobPosting`, `Application`, `User`.
- **JobApplicationController**
  `createJobApplication(userId, jobId, status)`; `updateJobApplication(appId, status, note)`; `listJobApplications(userId)`; manages lifecycle of `Application`.
- **ProfileController**
  `getProfile(userId)`; `updateProfile(...)`; `addSkill(...)`; `removeSkill(...)`; manages `User`, `UserSkill`.

- **AuthenticationController**
  `login(email, password)`; `register(userData)`; `logout(sessionId)`; returns/invalidates `sessionId` and errors; accesses `User`.

- **RecommendationController**
  `getRecommendations(userId)`; `computeMatch(resume, jobSkills)`; `rankJobs(scores)`; writes `Recommendation(userId, jobId)`.

- **CourseController**
  `getCourses()`; maps `JobSkill` gaps to `Course` via `CourseSkill`.

2.2.2.3 App Object Layer (Entity Classes)

- **User** (`userId`, `name`, `email`, `passwordHash`, `passwordSalt`, `school`, `gradYear`) – has many `UserSkill`, `Application`, `Recommendation`.

- **Skill** (`skillId`, `name`) – linked to `User` via `UserSkill` and to `JobPosting` via `JobSkill`.

- **UserSkill** (`userId`, `skillId`, `proficiency`, `source`).

- **JobPosting** (`jobId`, `title`, `company`, `location`, `description`, `postedAt`, `applyUrl`, `active`) – has many `Application` and `JobSkill`.

- **JobSkill** (`jobId`, `skillId`, `weight`) – required skills per job.

- **Sector** (`sectorId`, `name`, `vacancyIndex`, `growthDirection`) – ranked by **SectorHotSkill**.

- **SectorHotSkill** (`sectorId`, `skillId`, `rank`) – hotness ranking inside a sector.

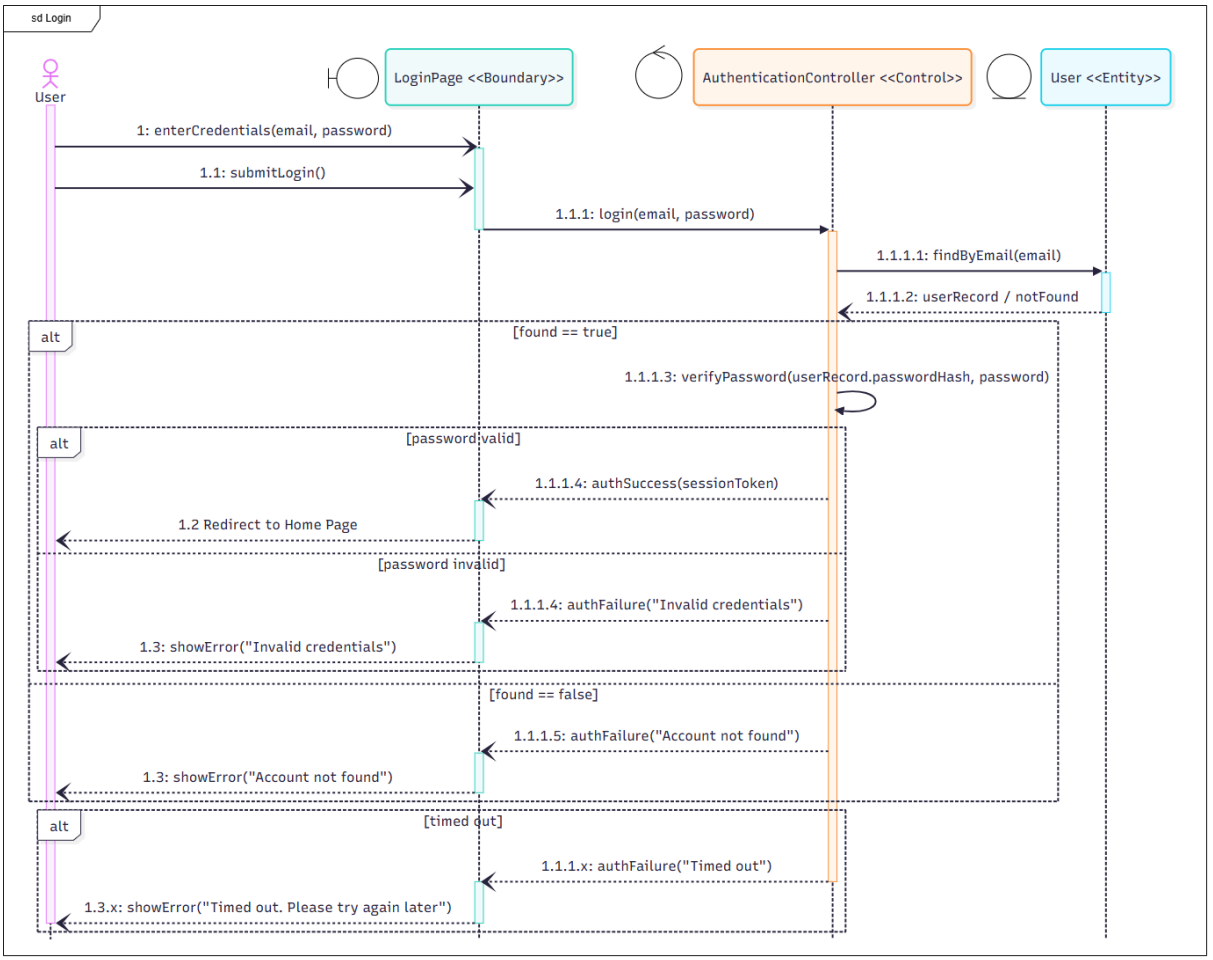
- **JobApplication** (`appId`, `userId`, `jobId`, `status`, `note`, `updatedAt`).

- **Recommendation** (`userId`, `jobId`) – a 1..* for user to ranked jobs.

- **Course** (`courseId`, `title`, `provider`, `url`) – mapped to skills via **CourseSkill**.
- **CourseSkill** (`courseId`, `skillId`) – what a course teaches.

2.2.3 Sequence Diagrams

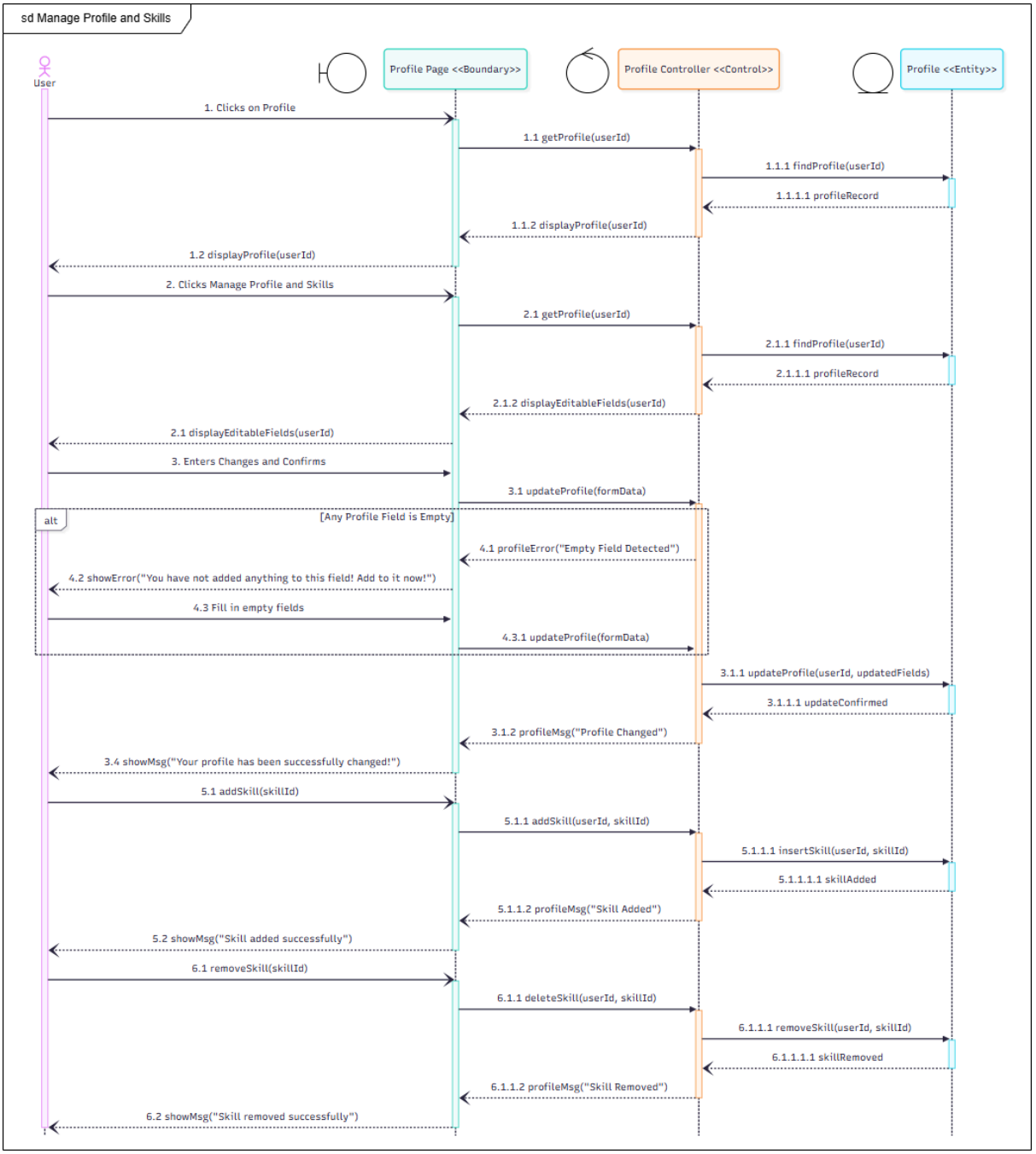
2.2.3.1 Use Case - UC-2 (Login)



Description:

Users log in to the system by inputting their email and password. If their account email is found in the system records, the system will check their password to see if it is correct. If correct, the user is redirected to the Home Page. Otherwise, the system informs them that the password is incorrect. If the email is not found in the records, the system informs them that the account does not exist. If the system times out and fails the authentication of the email and password, the system sends an error message and informs the user.

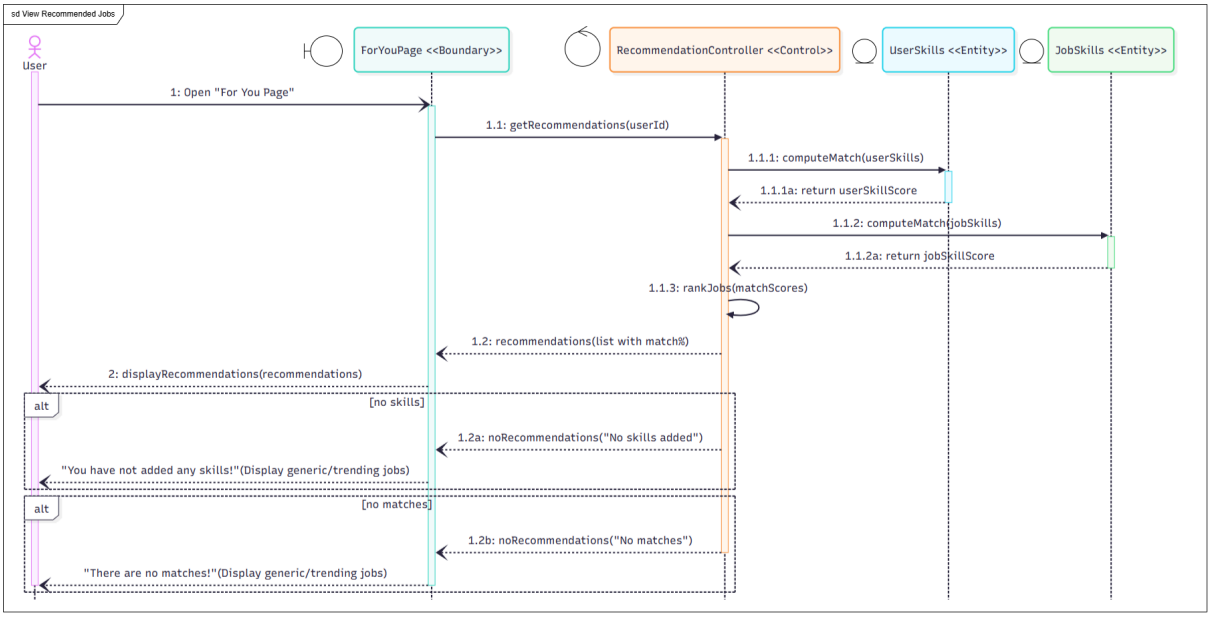
2.2.3.2 Use Case - UC-3 (Manage Profile and Skills)



Description:

The user clicks on the profile page to view it. On the page, there is an option to manage profile and skills which allows them to adjust the profile fields. After they input their changes, they can submit them to the system. The system checks the profile fields and confirms their updates if they are filled up. Otherwise if it detects an empty field, it prompts the user to fill them up. After they have been filled up, the system updates and confirms the changes, informing the user the changes to their profile have been made successfully. The user can also add and delete skills from their profile.

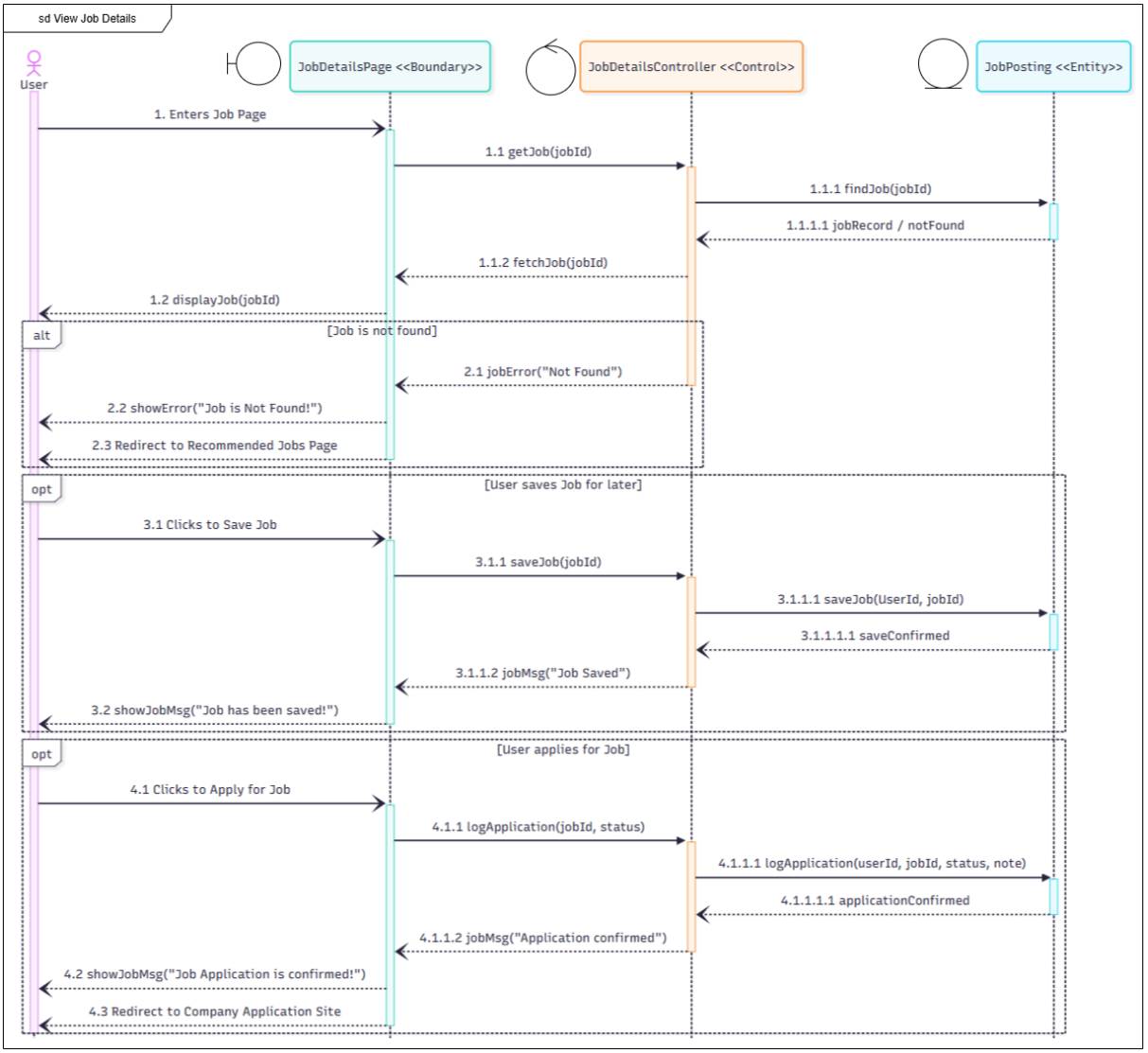
2.2.3.3 Use Case - UC-5 (View Recommended Jobs)



Description:

User clicks on the For You page, which shows recommended jobs to the user based on the skills in their profile. The system retrieves the jobs by matching the user skills to the job requirements and displaying them to the user on the page when a match is found. If the user has no skills in their profile or there are no matches which brings up no recommendations, the user is informed that they have no skills added and the system instead displays generic or trending jobs.

2.2.3.4 Use Case - UC-7 (View Job Details)

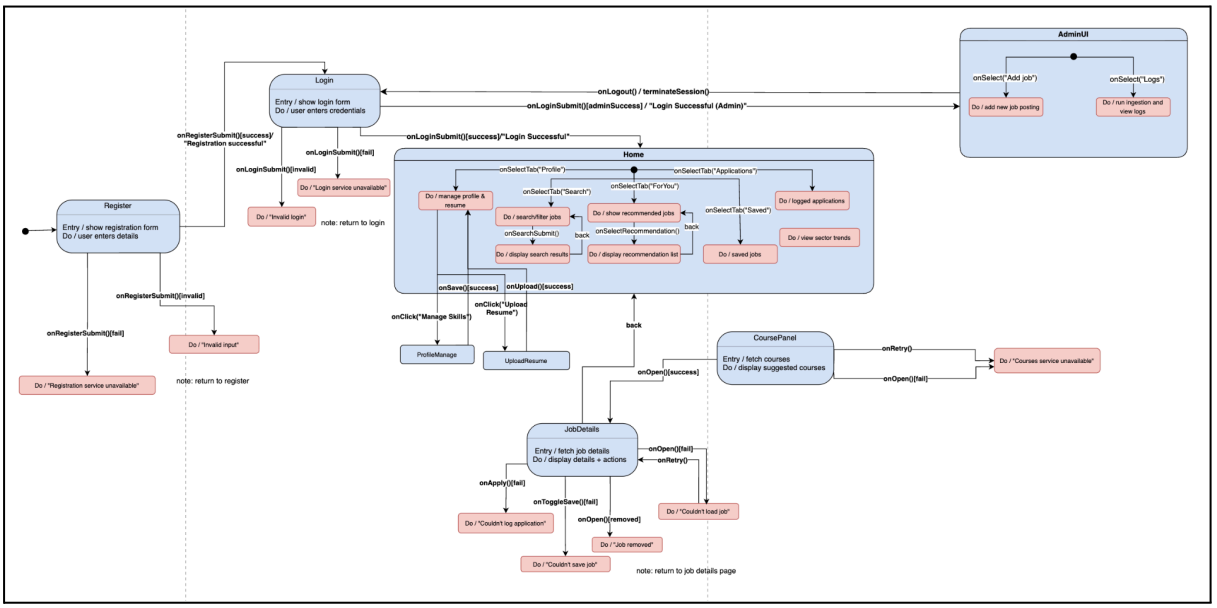


Description:

The user enters the Job page, which displays the details of the job that the system has retrieved for the user. If the job is found, it is displayed to the user on the page. Otherwise, the system informs the user the job is not found and redirects them to the Recommended Jobs page. The user has the option to save the job for later when they want to have another look at it another time. They also have the option to apply for the job which logs their application in the system, recording and informing the user that they have applied for the job. After which, they are redirected towards the company's application site.

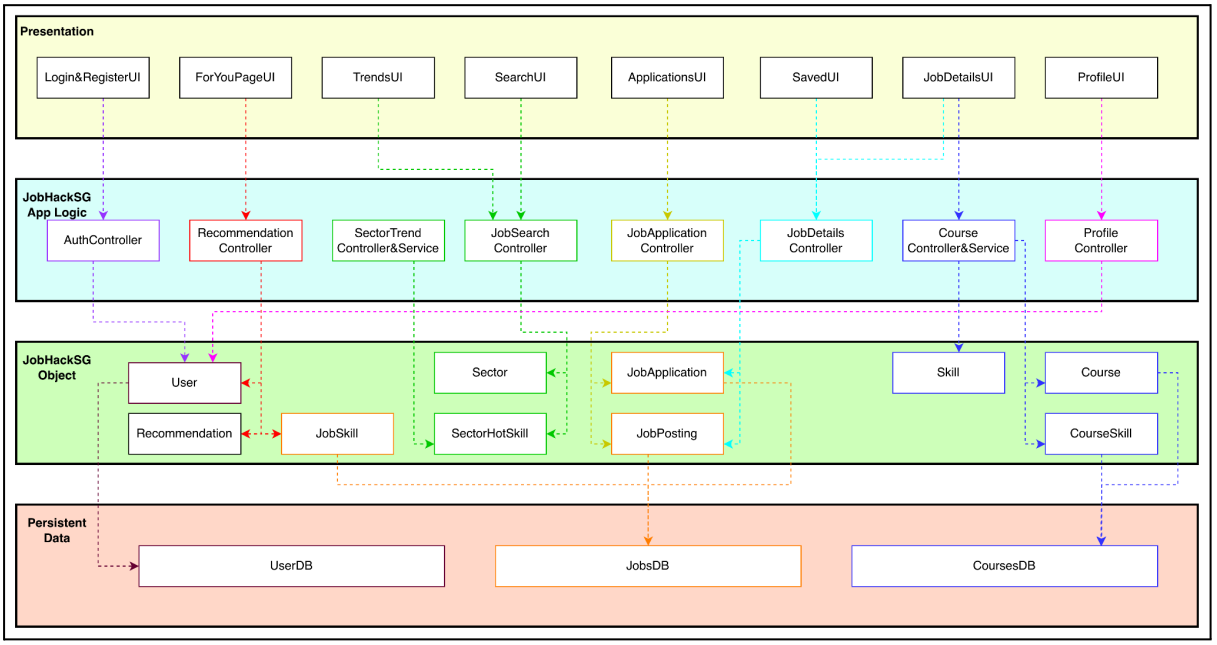
2.2.4 Dialog Map Diagram

If the image is unclear, please refer to the separate file on GitHub.



2.3 System Architecture

If the image is unclear, please refer to the separate file on GitHub.



2.3.1 Presentation

This layer handles the interaction between the users and the application interface. It involves the UI components of the app that the user interacts with.

UI Components:

- `Login&RegisterUI`: Manages user login and registration.
- `ForYouPageUI`: Displays personalized job recommendations.
- `TrendsUI`: Shows trending sectors and skills.
- `SearchUI`: Allows users to search for job postings.
- `ApplicationsUI`: Displays the applications made by the user.
- `SavedUI`: Displays saved jobs for easy access.
- `JobDetailsUI`: Shows detailed information about the selected job.
- `ProfileUI`: Displays and manages user profile information.

2.3.2 JobHackSG App Logic Layer:

This layer contains the logic that controls the functionality and flow of the application. Controllers manage the requests between the UI and the backend services.

Key Controllers:

- `AuthController`: Handles user authentication.
- `Recommendation Controller`: Helps in fetching job recommendations based on user profile and skills.
- `SectorTrend Controller&Service`: Fetches and processes sector trends.
- `JobSearch Controller`: Handles job search functionality.
- `JobApplication Controller`: Manages job application processes.
- `JobDetails Controller`: Retrieves detailed information for job postings.
- `Course Controller&Service`: Helps in providing recommendations for courses based on skill gaps.
- `Profile Controller`: Manages user profile and skill updates.

2.3.3 JobHackSG Object Layer

This layer includes the entities and objects that interact with the database and represent real-world objects in the application. *Note: Not all entity classes from the class diagram are included here, because this is mainly representing the system level architecture.*

Main Entities:

- `User`: Represents the user of the system, including their details.
- `JobApplication`: Represents a job application submitted by a user.
- `Sector`: Represents job sectors and trends.
- `Skill`: Represents the skills relevant to users and job postings.
- `Course`: Represents available courses to fill skill gaps.
- `JobPosting`: Represents job listings that are available for application.
- `Recommendation`: Represents personalized job recommendations.

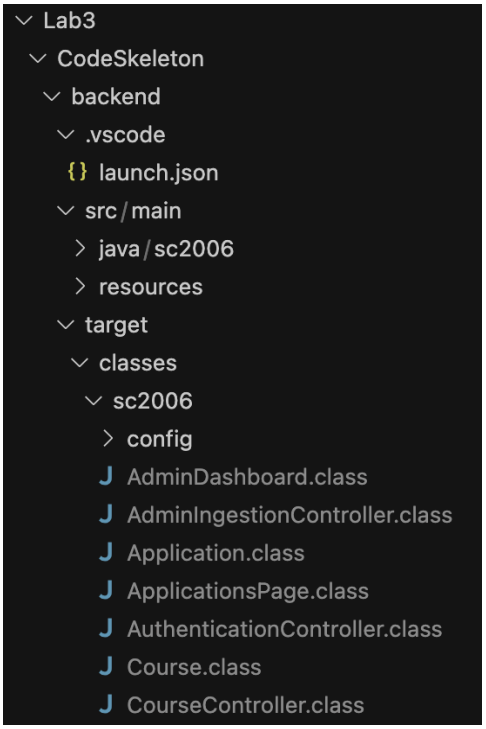
2.3.4 Persistent Data Layer

This layer handles the data storage and retrieval from the database. It includes the databases that store the system's data.

Databases:

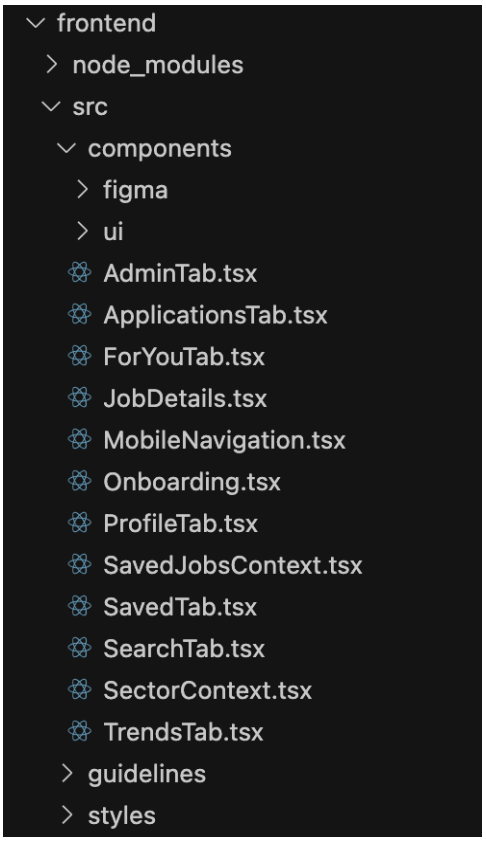
- `UserDB`: Stores user data.
- `CoursesDB`: Stores courses available for users to improve their skills.
- `JobsDB:` Contains a list of jobs posted by companies.

2.4 Application Skeleton



Backend:

The backend is developed using Java Spring Boot and follows a layered architecture consisting of controllers, services, repositories, and models. Each layer has a distinct role to ensure modularity, maintainability, and clarity in data flow. The controllers define REST API endpoints (e.g., /courses, /jobs, /applications) and handle incoming HTTP requests, returning responses in JSON format. The services implement the main business logic, handling data processing, validation, and interaction between the controllers and repositories. The repositories manage communication with the database, typically by extending Spring Data JPA interfaces for CRUD operations. The models define entity classes such as Course, Job, User, and Application, which represent the structure of database tables. The backend's main entry point, Application.java, is annotated with @SpringBootApplication and is responsible for bootstrapping the application and exposing the REST endpoints.



Frontend:

The frontend is built using React with TypeScript and structured under the frontend/src directory.

It primarily consists of multiple screens (UI components), which are responsible for rendering different user interfaces such as job listings, course information, and application management.

The App.tsx is the entry point of the frontend application. Other folders contain reusable UI elements like buttons that are shared among different screens, distinct sub-screens for different user roles or functions, contains TypeScript helper files (e.g., courses.ts, applications.ts) that define the methods for calling backend REST APIs using fetch or axios and includes utility/helper functions such as date formatters, validation methods, or constants used across multiple screens.

2.5 Operating Environment

The application will be **web-based** and accessed through a **modern browser**. It is expected to run on the latest stable versions of major browsers, including **Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari**. Compatibility should follow current HTML5, CSS3, and ES6 standards so that no browser-specific plugins are required. Where browser-specific behavior exists (e.g. Safari security policies), appropriate fallbacks or polyfills should be implemented.

On the host/developer environment, the **operating system must support both Java/Maven and Node.js** toolchains, since the application stack involves a Java-based backend and a JavaScript-based frontend. That means the OS should be one of:

- Windows 10/11 (with Java 17+ or whatever version the project uses, plus Maven, plus Node.js/NPM)
- macOS (current stable, with Homebrew or equivalent to install Maven and Node.js)
- Linux distributions commonly used for development/deployment (e.g. Ubuntu LTS), with `openjdk`, `maven`, and `node` available via package managers

The typical workflow will be:

1. **Backend**: built and run using Maven commands such as `mvn clean install` and `mvn spring-boot:run`.
2. **Frontend**: built and served using Node.js commands such as `npm install` / `npm run dev`.

Because both toolchains are required, the environment must have:

- **Java SDK** installed (not just the JRE), matching the project's `pom.xml` requirements.
- **Apache Maven** available on the system PATH.
- **Node.js** (LTS) and **npm** installed for dependency management and building the frontend.
- Permissions to run these commands from the terminal/shell.

For deployment or CI/CD, the target server or pipeline must likewise support running Maven builds (for the backend artifact) and Node.js builds (for the frontend static assets) before packaging. In production, the delivered artifact will still be browser-accessed, but the build stage requires both runtimes.

3. UI Mockups

This is the UI mockup of our product based on what we have planned out in the prior section.

3.1 Onboarding

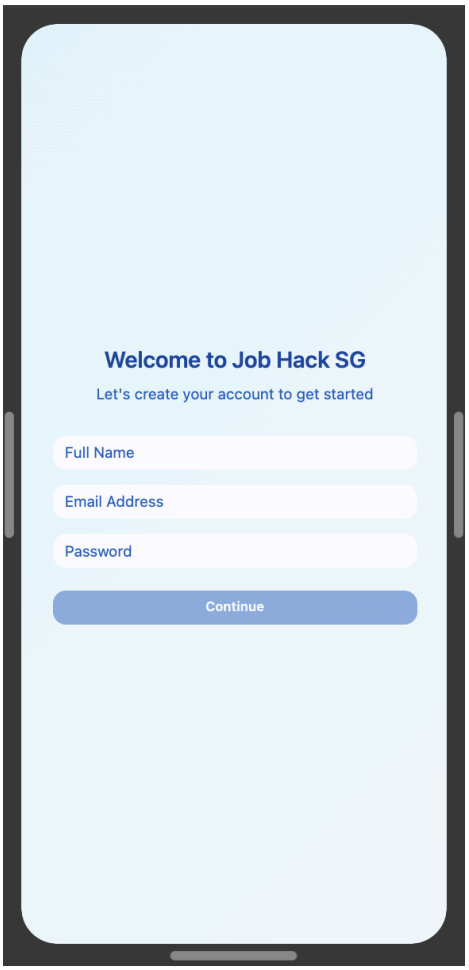


Figure 1.1

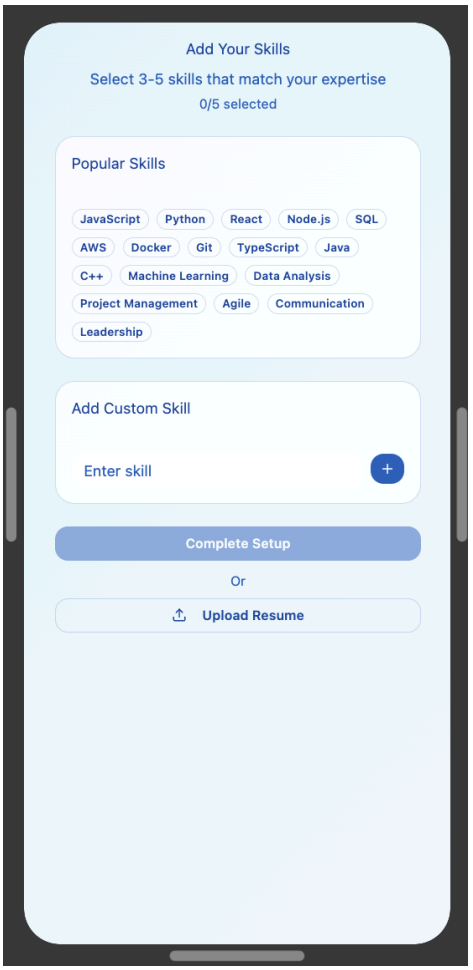


Figure 1.2

Figure 1.1 shows the Login Page of the application, where users are prompted to create a new account by providing their name, email address, and password. Figure 1.2 shows the Profile Creation Page, which enables users to select relevant skills from a predefined list of popular skills or add custom skills manually. Additionally, the application allows users to upload their resume to complete the account setup.

3.2 For You Page

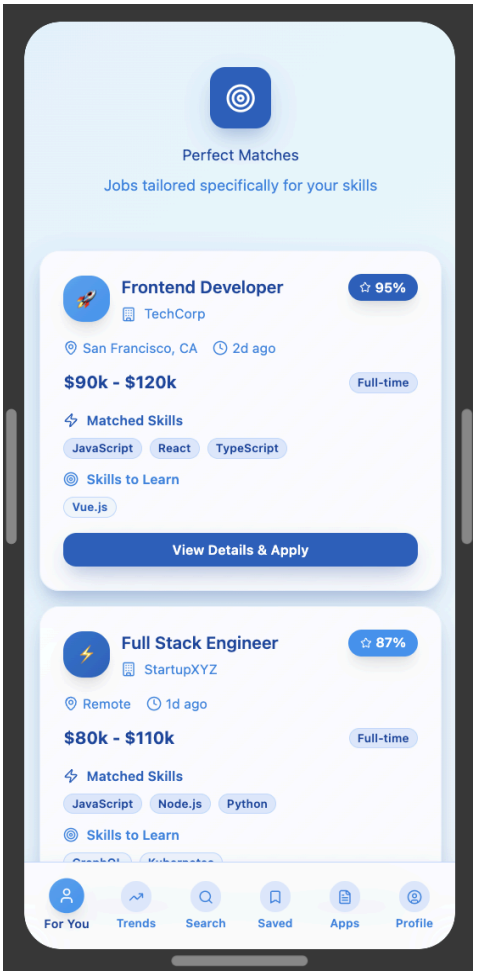


Figure 2.1

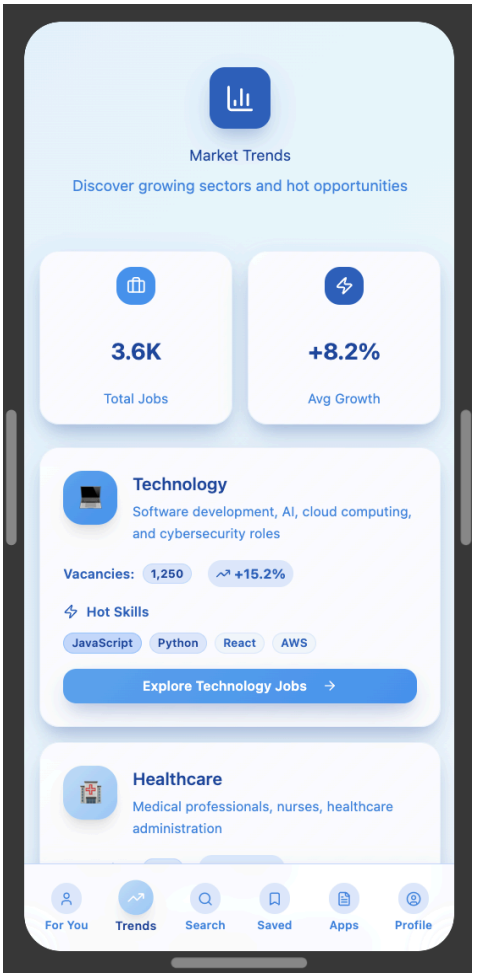


Figure 2.2

Figure 2.1 shows the Home Page, where job listings tailored to the user's profile are displayed. Figure 2.2 shows the Trends Page, where users can view job market trends categorized by different industries along with the popular skills each industry is looking for.

3.3 Search Page

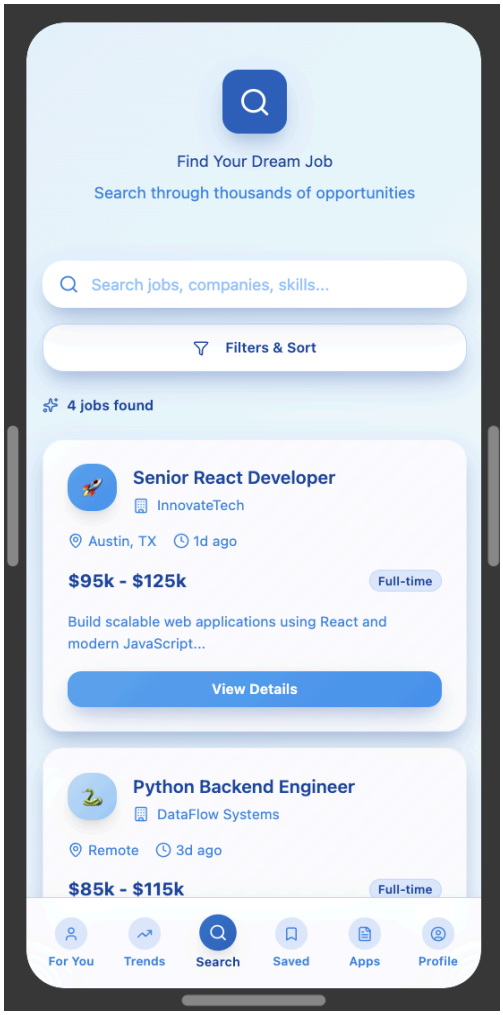


Figure 3.1

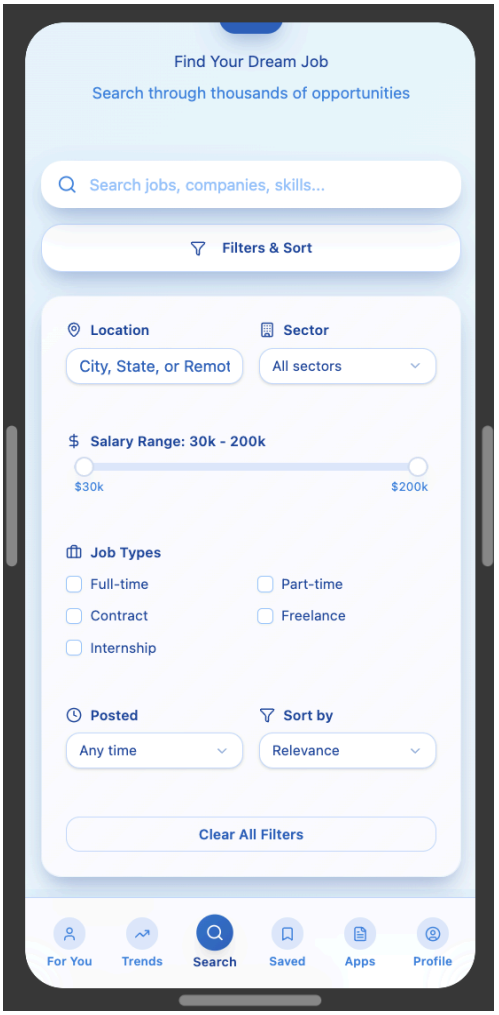


Figure 3.2

Figure 3.1 shows the job search page, which allows users to search for job opportunities that match their specific interests. Figure 3.2 shows the filter and sort functionality, enabling users to refine results by location, sector, salary range, and job type, as well as sort postings by the date they were advertised.

Note: While the prototype currently displays some sample jobs with overseas locations, the actual application is scoped to roles within Singapore only.

3.4 Job Details

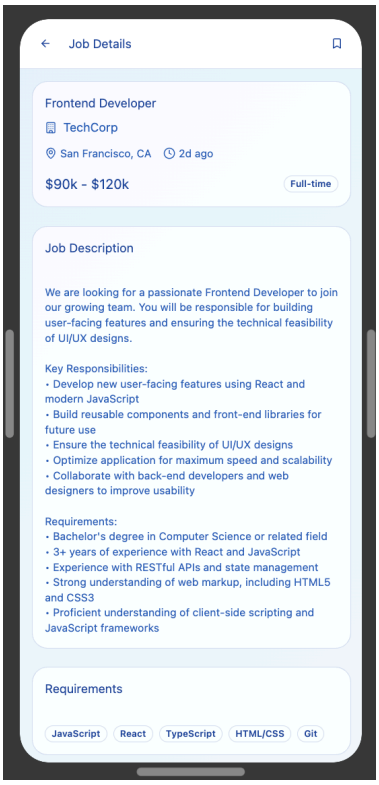


Figure 4.1

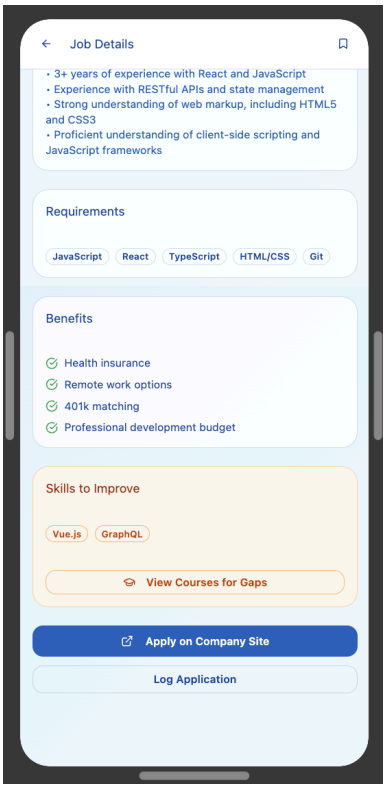


Figure 4.2

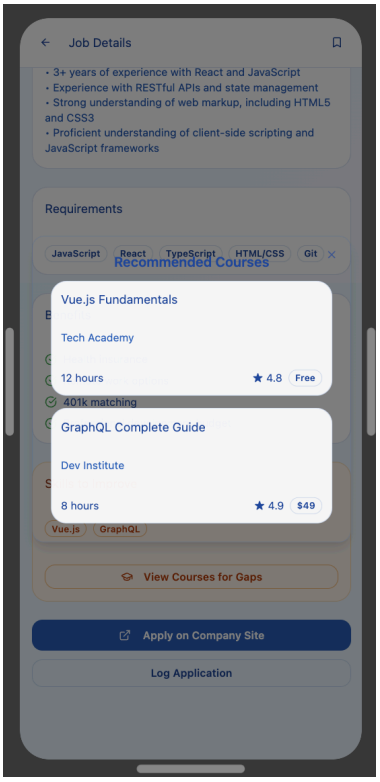


Figure 4.3

Fig 4.1 shows the Job Details page for a selected role. The page provides comprehensive information about the position, including the job title, company name, location, posting date, salary range and employment type. Users can read the full job description and key responsibilities of the job. Fig 4.2 details the required skills for the job, the benefits as well as highlighting “Skills to Improve” along with an option to view suggested courses to help bridge the gap. It also shows the interactive options provided by the application. Users can choose to save this job for further perusal by clicking on the top left “save” button on the page, apply directly through the company’s site, or log their application for that job so they have their own reference of it on this app using the buttons on the bottom of the page. Fig 4.3 shows samples of the suggested courses that users can undertake to bridge the gap between their skills and the company’s requirements, as well as other relevant information such as the academy offering the course, the rating, cost and time taken to complete the course.

3.5 Applications

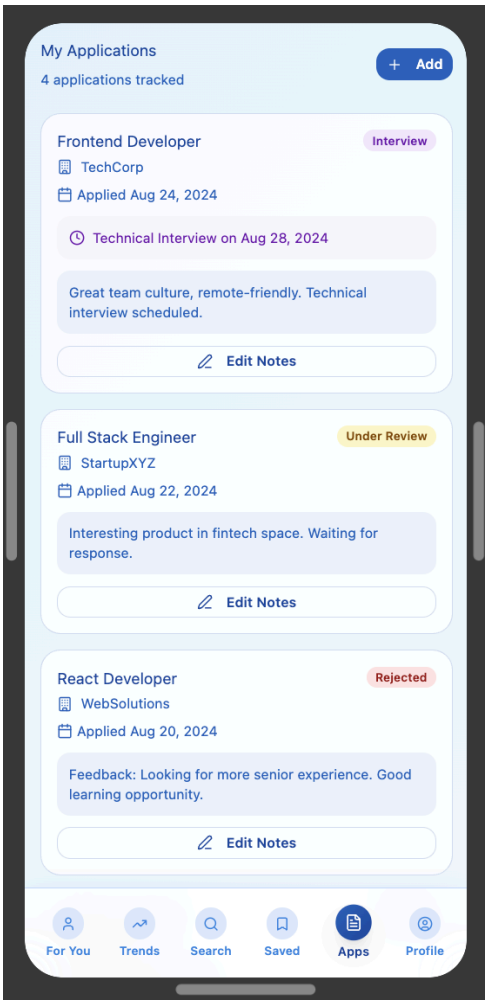


Figure 5.1

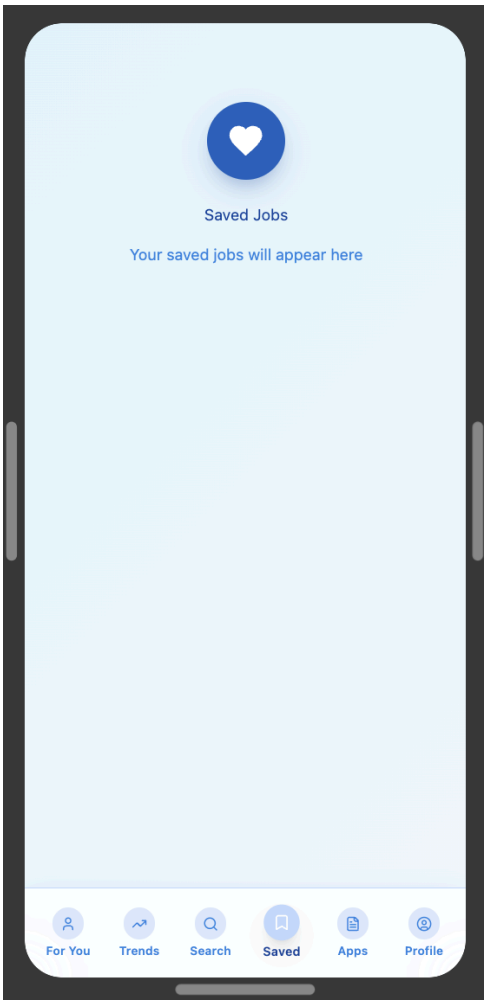


Figure 5.2

Fig 5.1 shows the My Applications page, where users can track all the roles they've applied for in one place. Each entry displays the job title, company name, application date, and current status - Applied, Under Review, Interview, Rejected. Additional details such as interview dates, recruiter feedback, or personal notes can be added and edited directly, allowing users to maintain a personalised record of their job search progress. Fig 5.2 shows the Saved page, which allows users to save jobs that they are interested in for future research.

Note: job applications are not submitted directly through this app; instead users apply directly via the respective company's site, and record the details here.

3.6 Profile

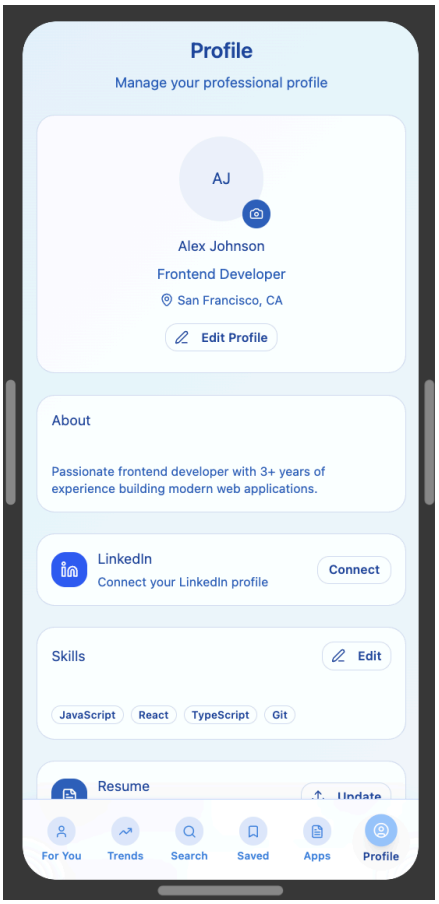


Figure 6.1

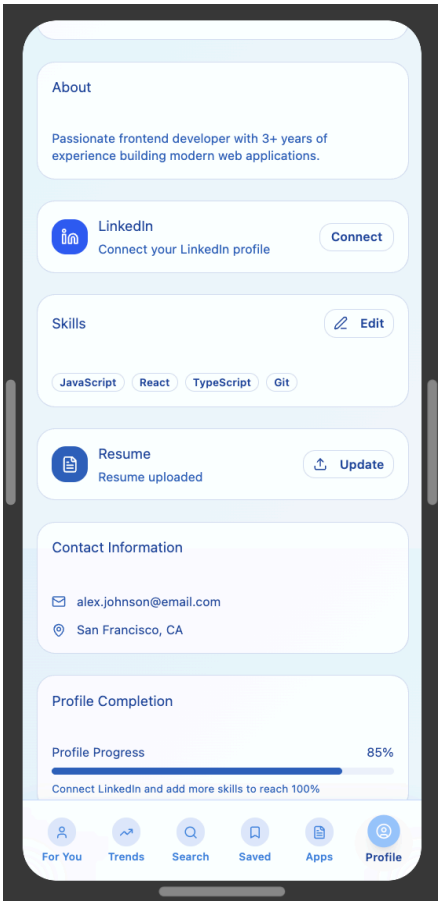


Figure 6.2

Fig 6.1 and 6.2 show the Profile page, where users manage their professional identity within the platform. Fig 6.1 showcases the user's basic details such as their name, role and location, as well as links to external profiles such as LinkedIn. It also highlights their list of Skills, with the ability to update it constantly if there are changes. Fig 6.2 has other key elements like resume upload and update functionality, and contact information for easy reference. A Profile Progress tracker motivates users to complete their profile by connecting LinkedIn or adding more skills.

Note: UC-11 is not included in the UI mockup because the mockups are intended to illustrate only the end-user's perspective, not the administrative or backend workflows. The functionality described in UC-11 is administrative in nature and falls outside the scope of the user-facing interface. Instead, UC-11 is documented in the system specification to ensure that the functionality is accounted for without cluttering the user-centric mockups

4. Testing

The section below details the testing of the prototype we have implemented, including the black and white box testing we have done.

4.1 Black Box Testing

4.1.2 AuthenticationController

Control Class to test - **AuthenticationController**

The AuthenticationController class is responsible for managing user authentication within the application, encompassing both user registration and user login functionalities.

The AuthenticationController is an integral component that facilitates secure interaction between the client and the authentication services of the system. When a user attempts to log in, the controller receives their login credentials, specifically the email and password. It then forwards these details to the AuthService for authentication. The system validates the provided credentials to ensure they match the stored records within the database. If the verification process confirms that the email exists and the password hash corresponds correctly to the stored hashed value, the login is deemed successful, and the user is granted access to the application. In this process, the controller also retrieves user information such as user ID, name, role, and associated skills to return a structured response to the client. The primary information managed by the AuthenticationController during user login includes the user's email, password hash, and related account details retrieved upon successful authentication.

In addition to login functionality, the AuthenticationController also manages user registration, which allows new users to create an account within the system. During registration, the controller collects essential information such as name, email, password, role, and a list of skills. The collected data is validated to ensure accuracy and completeness before being processed. The password provided by the user undergoes a secure hashing procedure prior to storage to maintain confidentiality and prevent unauthorized access. After validation, the controller communicates with the AuthService to store the new user's details in the database. If the user has provided a list of skills, these are subsequently added and linked to the registered account.

The AuthenticationController also handles special conditions within registration, such as checking for duplicate email entries or invalid input formats. In such cases, it returns appropriate error messages indicating the nature of the registration failure. Upon successful registration, a unique user ID is generated and assigned to the new user. This identifier becomes crucial for tracking user data and managing authentication across the application. Information managed by the AuthenticationController in the registration process includes name, email, password, role, school, graduation year, and skills.

4.1.2 Equivalence Class and Boundary Value Testing

Equivalence Class Testing

Equivalence Class Testing divides input data into valid and invalid partitions (or classes) that are expected to behave similarly. The technique aims to identify subsets of data that are logically equivalent in how the program processes them, allowing testers to select representative values from each class instead of testing every possible input. This partitioning is typically done based on the system's input specifications, constraints, and validation rules. Each equivalence class represents a range or set of conditions that share common characteristics, such as format, type, or range validity.

For instance, valid equivalence classes represent input data that meet all functional requirements and constraints of the system, while invalid equivalence classes contain data that violate one or more of these constraints. The division ensures that the testing process identifies potential defects arising from incorrect handling of valid inputs as well as inadequate rejection of invalid inputs.

By testing one representative from each class, redundancy is reduced while ensuring broad coverage of input possibilities.

Boundary Value Testing

Boundary Value Testing focuses on testing input values at the edges of allowed ranges. It helps detect off-by-one and input validation errors. This ensures the system behaves correctly under extreme but valid conditions, and fails gracefully when limits are exceeded.

1. Login function

Valid Equivalence Class: Emails and passwords with correct formats.
Invalid Equivalence Class: Emails and passwords with incorrect formats or missing information.

.

2. Register function

Valid Equivalence Class: Name, Email, Password, Role, Skills with correct formats.
Invalid Equivalence Class: Name, Email, Password, Role, Skills with incorrect formats or missing information.

4.1.3 Test Cases and Results

4.1.3.1 Login

Input parameters: Email and password

No.	Test Input	Expected Output	Actual Output	Pass?
1	**(Correct email and password)** **(Valid)** Email: “test123@gmail.com” **(Valid)** Password: “Password123”	Successful login	Successful login	YES
2	**(Valid email but wrong password)** **(Valid)** Email: “test123@gmail.com” **(Invalid)** Password: “Pass222222”	Login failed, system notify “Something went wrong during login. Please try again later.”	Login failed, system notify “Something went wrong during login. Please try again later.”	YES
3	**(Invalid email format)** **(Invalid)** Email: “test123” **(Valid)** Password: “Password123”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	YES
4	**(Empty string for email)** **(Invalid)** Email: “” **(Valid)** Password: “Password123”	Login failed, system notify “Email is required. Please fix the highlighted fields and try again”	Login failed, system notify “Email is required. Please fix the highlighted fields and try again”	YES
5	**(Empty password)** **(Valid)** Email: “test123@gmail.com” **(Invalid)** Password: “”	Login failed, system notify “Password is required. Please fix the highlighted fields and try again”	Login failed, system notify “Password is required. Please fix the highlighted fields and try again”	YES

4.1.3.2 Register

Input parameters: Name, Email, Password, Role, Skills

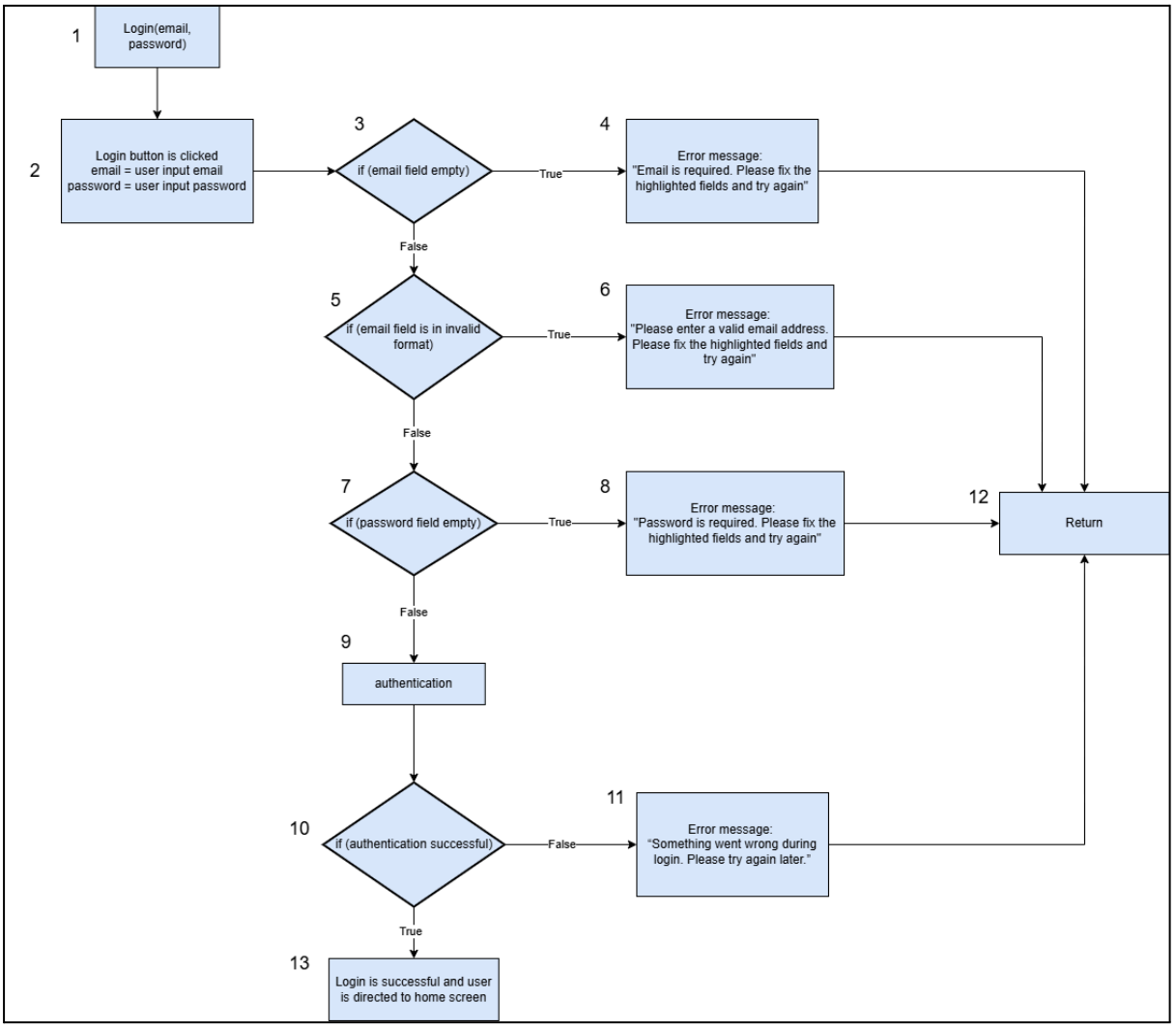
No.	Test Input	Expected Output	Actual Output	Pass?
1	**(All required fields provided correctly)** Name: “John Tester” Email: “test123@gmail.com” Password: “Password123” Role: “Fresh Graduate”	Successful registration	Successful registration	YES
2	**(Missing name)** Name: ””	Registration failed, system notify “Name is required”	Registration failed, system notify “Name is required”	YES
3	**(Missing email)** Email: “”	Registration failed, system notify “Please enter a valid email address”	Registration failed, system notify “Please enter a valid email address”	YES
4	**(Invalid email format)** Email: “test123”	Registration failed, system notify “Please enter a valid email address”	Registration failed, system notify “Please enter a valid email address”	YES
5	**(Missing password)** Password:””	Registration failed, system notify “Password must be at least 8 characters long”	Registration failed, system notify “Password must be at least 8 characters long”	YES
6	**(Invalid password format)** Password: “pass”	Registration failed, system notify “Password must be at least 8 characters long”	Registration failed, system notify “Password must be at least 8 characters long”	YES
7	**(Invalid password format)** Password: “password”	Registration failed, system notify “Password must contain at least one uppercase letter”	Registration failed, system notify “Password must contain at least one uppercase letter”	YES
8	**(Duplicate email registration)**	Registration failed, system notify “Registration failed: Email already registered”	Registration failed, system notify “Registration failed: Email already registered”	YES
9	**(Missing optional fields)**	Registration failed,	Registration failed,	YES

	Role:””	system notify “Please select your current role”	system notify “Please select your current role”	
10	**(Less than 2 skills selected)**	Registration failed, system notify “Please select at least 2 skills”	Registration failed, system notify “Please select at least 2 skills”	YES

4.2 White Box Testing

4.2.1 Login

4.2.1.1 Control Flow Graph



4.2.1.2 Basis Path Testing

Cyclomatic Complexity: |decision points| + 1 = 4 + 1 = 5

Basis Paths:

1. Baseline path: 1, 2, 3, 5, 7, 9, 10, 13
2. Basis path #2: 1, 2, 3, 4, 12
3. Basis path #3: 1, 2, 3, 5, 6, 12
4. Basis path #4: 1, 2, 3, 5, 7, 8, 12
5. Basis path #5: 1, 2, 3, 5, 7, 9, 10, 11, 12

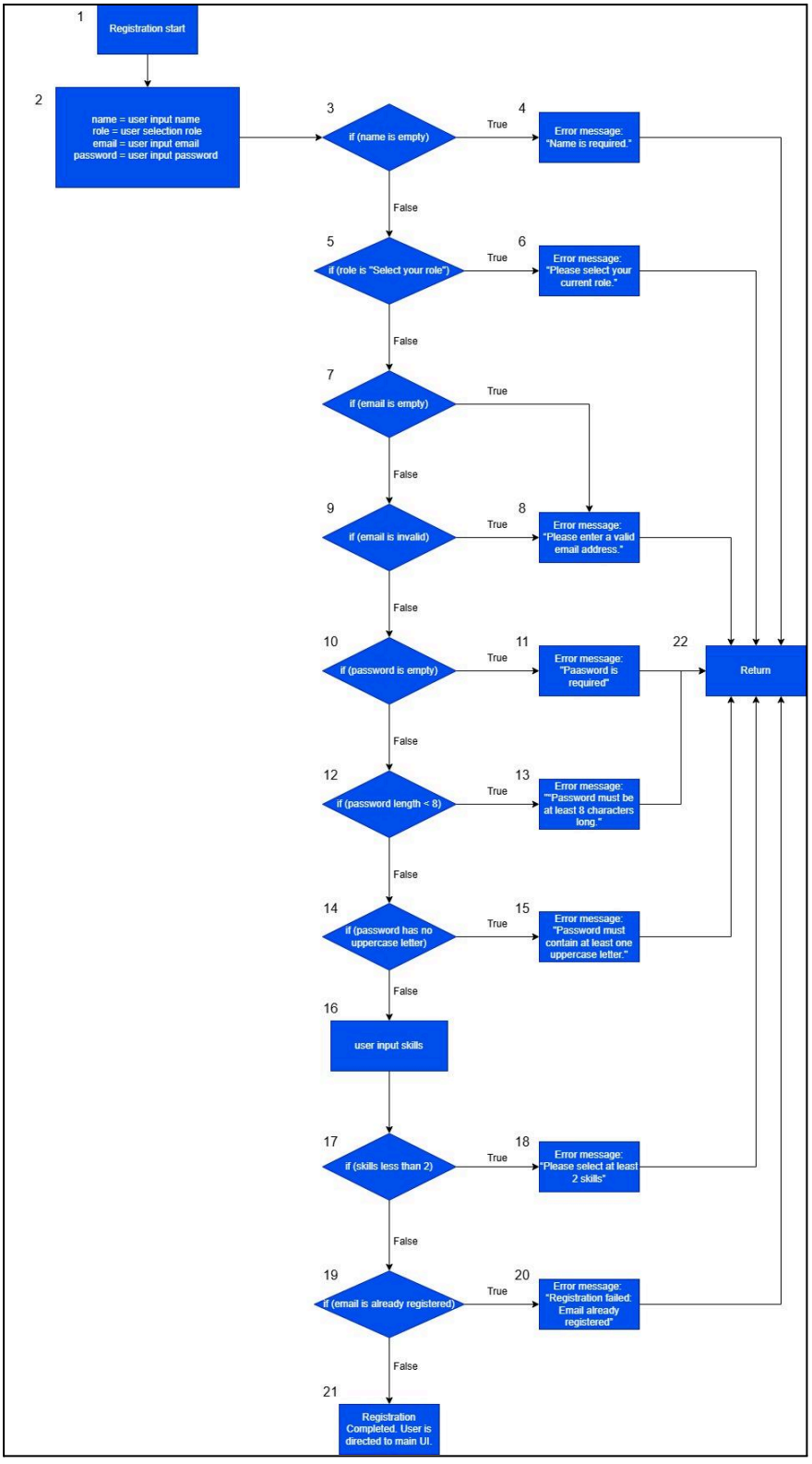
4.2.1.3 Test Cases and Results

No.	Test Input	Expected Output	Actual Output	Pass?
1	Email = “test123@gmail.com” Password = “Password123”	Successful login	Successful login	YES
2	Email = “” Password = “Password123”	Login failed, system notify “Email is required. Please fix the highlighted fields and try again”	Login failed, system notify “Email is required. Please fix the highlighted fields and try again”	YES
3	Email = “test123” Password = “Password123”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	YES
4	Email = “test123@” Password = “Password123”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	YES
5	Email = “test123@gmail.com” Password = “”	Login failed, system notify “Password is required. Please fix the highlighted fields and try again”	Login failed, system notify “Password is required. Please fix the highlighted fields and try again”	YES
6	Email = “test123@gmail.com” Password = “WrongPass222”	Login failed, system notified “Something went wrong during login. Please try again later.”	Login failed, system notified “Something went wrong during login. Please try again later.”	YES

7	Email = “wrong123@gmail.com” Password = “Password123”	Login failed, system notified “Something went wrong during login. Please try again later.”	Login failed, system notified “Something went wrong during login. Please try again later.”	YES

4.2.2 Registration

4.2.2.1 Control Flow Graph



4.2.2.2 Basis Path Testing

Cyclomatic Complexity: |decision points| + 1 = 9 + 1 = 10

Basis Paths:

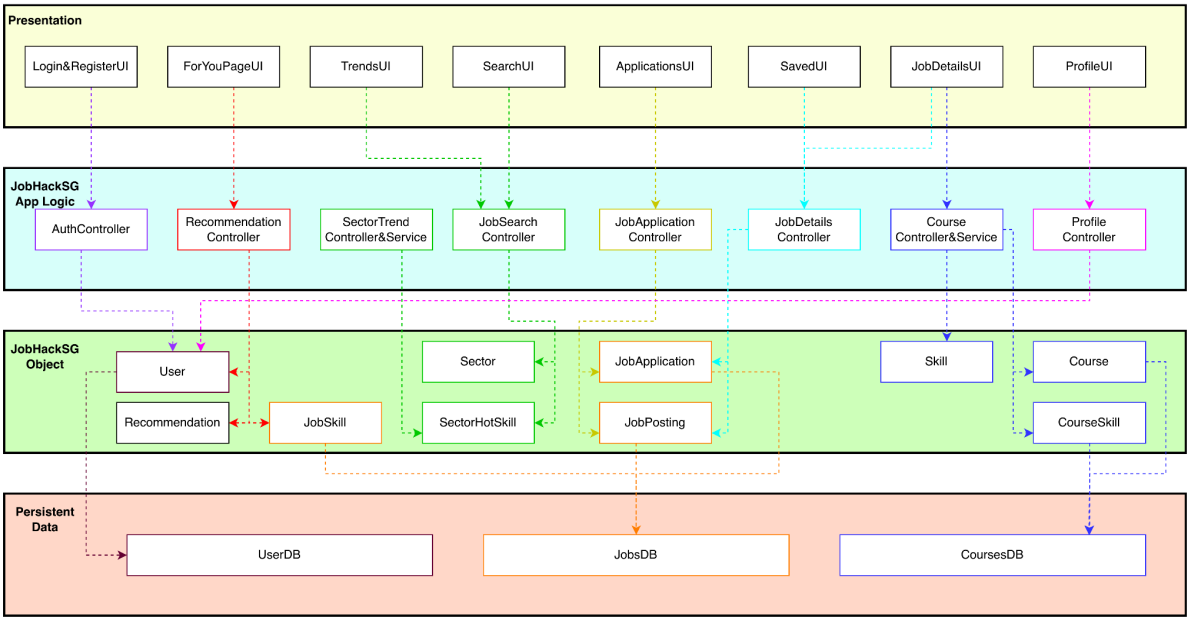
1. Baseline path: 1, 2, 3, 5, 7, 9, 10, 12, 14, 15, 17, 19, 21
2. Basis path #2: 1, 2, 3, 4, 22
3. Basis path #3: 1, 2, 3, 5, 6, 22
4. Basis path #4: 1, 2, 3, 5, 7, 8, 22
5. Basis path #5: 1, 2, 3, 5, 7, 9, 8, 22
6. Basis path #6: 1, 2, 3, 5, 7, 9, 10, 11, 22
7. Basis path #7: 1, 2, 3, 5, 7, 9, 10, 12, 13, 22
8. Basis path #8: 1, 2, 3, 5, 7, 9, 10, 12, 14, 15, 22
9. Basis path #9: 1, 2, 3, 5, 7, 9, 10, 12, 14, 15, 17, 18, 22
10. Basis path #10: 1, 2, 3, 5, 7, 9, 10, 12, 14, 15, 17, 19, 20, 22

4.2.2.3 Test Cases and Results

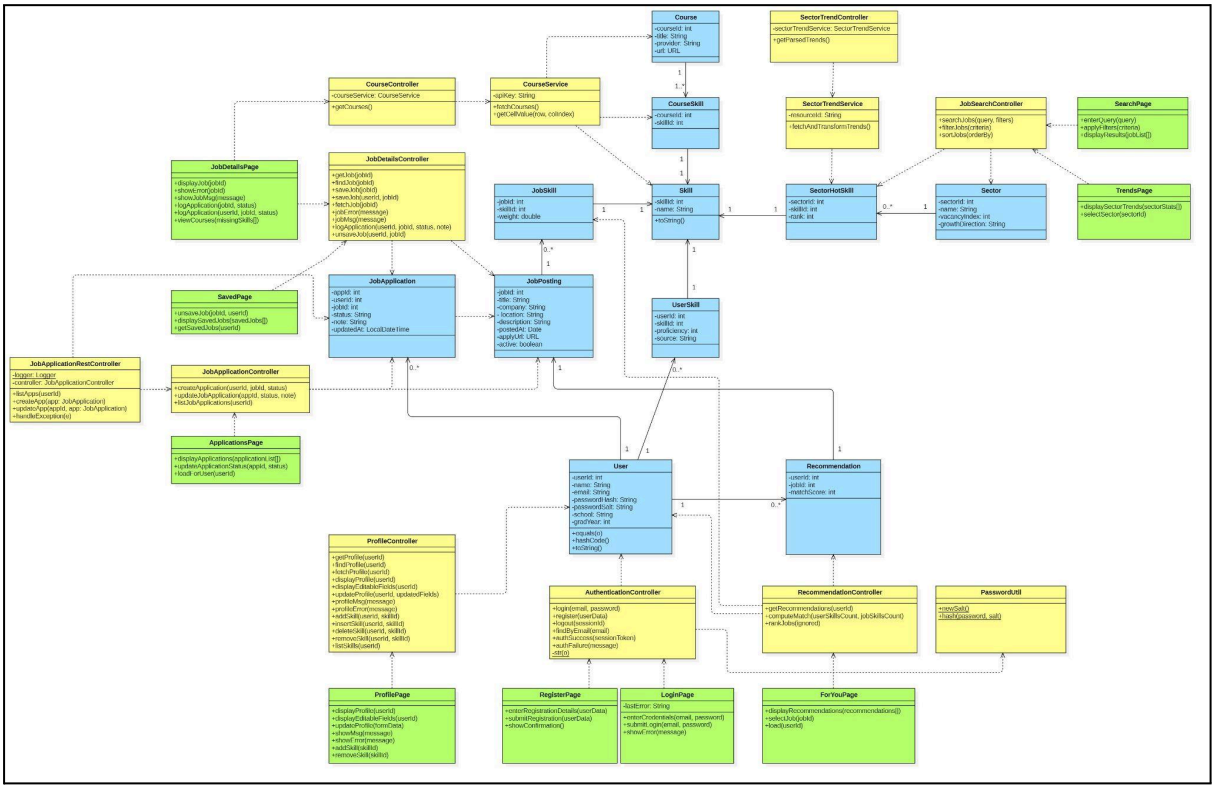
No.	Test Input	Expected Output	Actual Output	Pass?
1	name="John", email="john@gmail.com", passwordHash="Password123", role="Fresh Graduate", skills=["Java","Python"]	Registration successful, system displays main UI.	Registration successful, system displayed main UI.	YES
2	name="", email="jane@gmail.com", password="Password123", role="Fresh Graduate", skills=["Java", "Python"]	Exception raised, system notifies "Name is required."	Registration failed, system notified "Name is required." Cannot proceed to the skill selection page.	YES
3	name="Jane", email="jane@gmail.com", passwordHash="Password123", role="Select your role", skills=["Java", "Python"]	Exception raised, system notifies "Please select your current role."	Registration failed, system notified "Please select your current role." Cannot proceed to the skill selection page.	YES
4	name="Jane", email="", password="Password123", role="Fresh Graduate", skills=["Java", "Python"]	Exception raised, system notifies "Please enter a valid email address."	Registration failed, system notified "Please enter a valid email address." Cannot proceed to the skill selection page.	YES

5	name="Jane", email="jane@g", password="Password123", role="Fresh Graduate", skills=["Java”, “Python"]	Exception raised, system notifies “Please enter a valid email address.”	Registration failed, system notified “Please enter a valid email address.” Cannot proceed to the skill selection page.	YES
6	name="Jane", email="john@gmail.com", password="Password123", role="Fresh Graduate", skills=["Java”, “Python"]	Exception raised, system notifies “Registration failed: Email already registered.”	Registration failed, system notified “Registration failed: Email already registered” after skill selection.	YES
7	name="Jane", email="jane@gmail.com", password="", role=”Fresh Graduate”, skills=["Java”, “Python"]	Exception raised, system notifies “Password is required.”	Registration failed, system notified “Password is required.” Cannot proceed to the skill selection page.	YES
8	name="Jane", email="jane@gmail.com", password="123", role=”Fresh Graduate”, skills=["Java”, “Python"]	Exception raised, system notifies “Password must be at least 8 characters long.”	Registration failed, system notified “Password must be at least 8 characters long.” Cannot proceed to the skill selection page.	YES
9	name="Jane", email="jane@gmail.com", password="12345678", role=”Fresh Graduate”, skills=["Java”, “Python"]	Exception raised, system notifies “Password must contain at least one uppercase letter.”	Registration failed, system notified “Password must contain at least one uppercase letter.” Cannot proceed to the skill selection page.	YES
10	name="Jane", email="jane@gmail.com", password="Password123", role=”Fresh Graduate”, skills=[]	Exception raised, system notifies “Please select at least 2 skills.”	Registration failed, system notified “Please select at least 2 skills” at the skill selection page.	YES

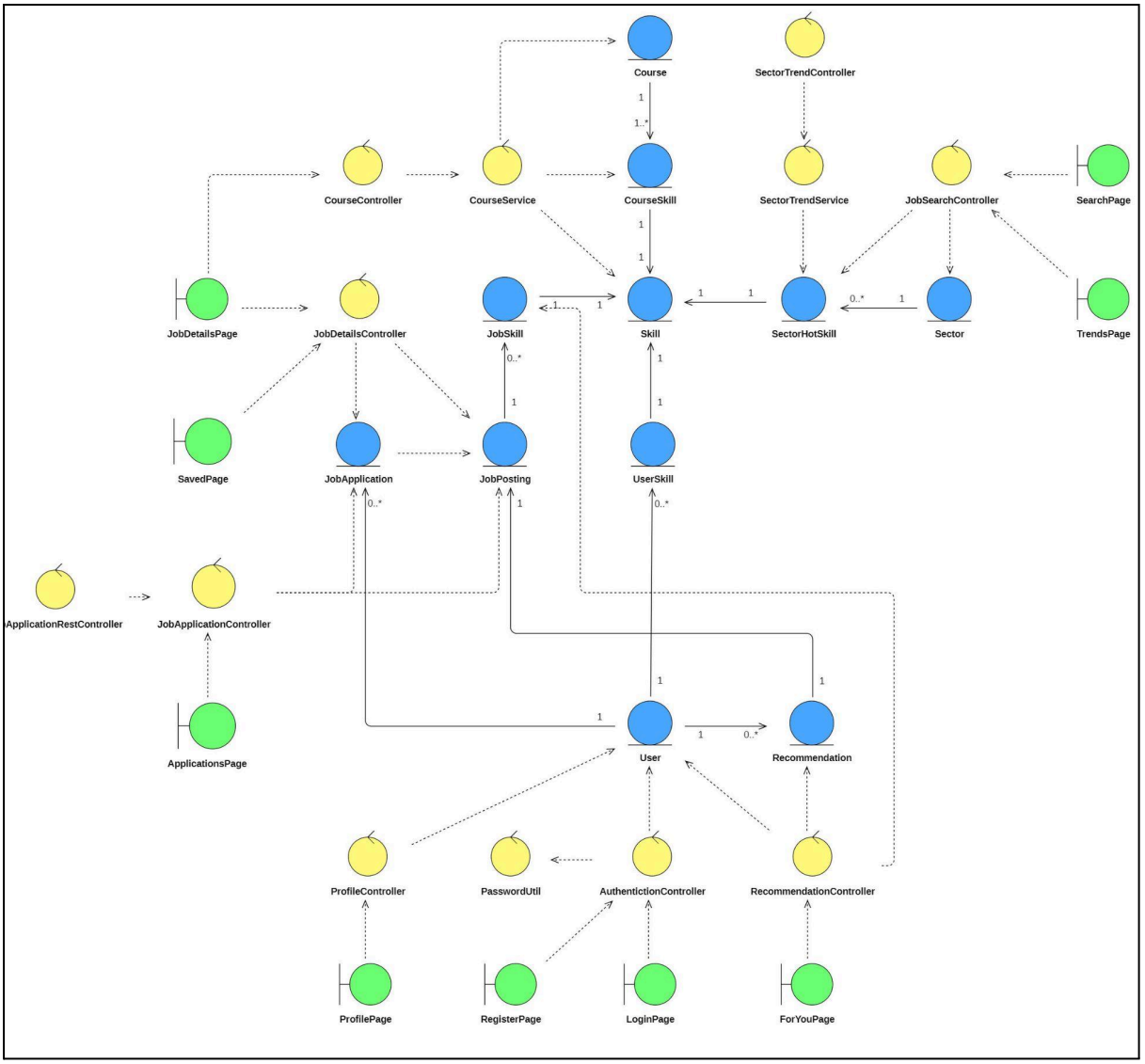
Appendix A: Analysis Models



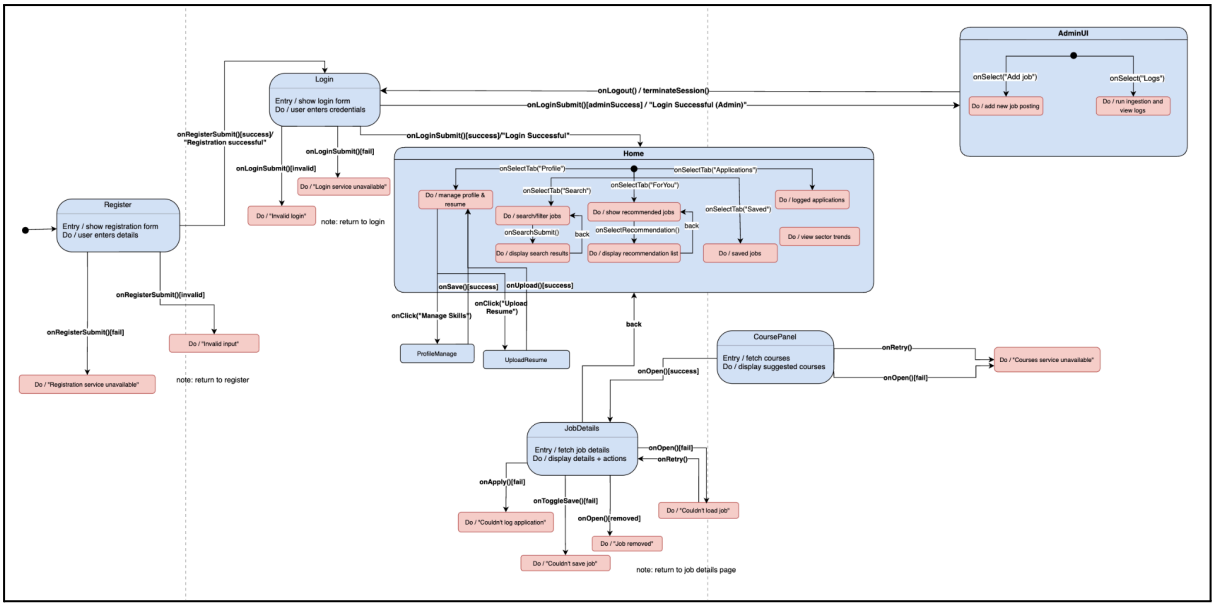
System Architecture Diagram



Class Diagram



Stereotype Class Diagram


AdminUI
onSelect("Add job")
onSelect("Logs")
Do / add new job posting
Do / run ingestion and view logs
Login
Entry / show login form
Do / user enters credentials
onLogout() / terminateSession()
onLoginSubmit()[adminSuccess] / "Login Successful (Admin)"
onRegisterSubmit()[success]/ "Registration successful"
onLoginSubmit()[fail]
onLoginSubmit()[success]/"Login Successful"
onLoginSubmit()[invalid]
Do / "Login service unavailable"
Do / "Invalid login"
note: return to login
Home
onSelectTab("Profile")
onSelectTab("Applications")
onSelectTab("Search")
onSelectTab("ForYou")
onSelectTab("Saved")
Do / manage profile & resume
Do / search/filter jobs
Do / show recommended jobs
Do / logged applications
back
onSearchSubmit()
onSelectRecommendation()
Do / view sector trends
Do / display search results
Do / display recommendation list
Do / saved jobs
Register
Entry / show registration form
Do / user enters details
onRegisterSubmit()[invalid]
onRegisterSubmit()[fail]
Do / "Invalid input"
Do / "Registration service unavailable"
note: return to register
onSave()[success]
onUpload()[success]
onClick("Manage Skills")
onClick("Upload Resume")
ProfileManage
UploadResume
back
CoursePanel
Entry / fetch courses
Do / display suggested courses
onRetry()
onOpen()[fail]
Do / "Courses service unavailable"
onOpen()[success]
JobDetails
Entry / fetch job details
Do / display details + actions
onOpen()[fail]
onRetry()
onApply()[fail]
onToggleSave()[fail]
onOpen()[removed]
Do / "Couldn't log application"
Do / "Couldn't load job"
Do / "Job removed"
Do / "Couldn't save job"
note: return to job details page

Dialog Map